MH_twork02_prob01

November 25, 2025

1 Introduction to Meta-heuristics: Team Work II - TSP problem

1.1 Team Members

1. m5291087 Tithira Withanaarachchi
2. m5291501 Minnah Aamir
3. s1300030 Kohei Kuwana

Role of Each Member.

- **Kohei Kuwana:** Apply GA to the TSP Problem: Includes coding, testing, Creating Presentation slides, Reviewing Report and Presentation slides.
- **Minnah Amir:** Apply ACO to TSP problem: Includes coding, testing, Creating Presentation Slides, Reviewing Report and Presentation slides.
- **Tithira Withanaarachchi:** Co-ordinating work, Parameter Optimization of the application of GA and ACO algorithms to TSP Problem: Coding, testing, Documenting with explanations, Creating Visuals of the Results, Creating Presentation slides, Reviewing Report and Presentation slides.

2 Problem Introduction

2.1 Traveling Salesman Problem (TSP)

TSP is a problem that seeks to find the route with the minimum total distance traveled (or cost) among all possible routes that visit each of multiple cities exactly once and return to the starting point.

https://commons.wikimedia.org/wiki/File:Example_The_travelling_salesman_problem_(TSP).gif

The method of obtaining the optimal solution by calculating all paths has a time complexity of O(n!) and is NP-hard. We will consider about n=10, 50 100.

3 Application of Algorithms

3.1 Initialization

[ ]:
```
pip install optuna
```

```
Requirement already satisfied: optuna in /usr/local/lib/python3.12/dist-packages
(4.6.0)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.12/dist-
packages (from optuna) (1.17.2)
Requirement already satisfied: colorlog in /usr/local/lib/python3.12/dist-
packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from optuna) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from optuna) (25.0)
Requirement already satisfied: sqlalchemy>=1.4.2 in
/usr/local/lib/python3.12/dist-packages (from optuna) (2.0.44)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from optuna) (4.67.1)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages
(from optuna) (6.0.3)
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-packages
(from alembic>=1.5.0->optuna) (1.3.10)
Requirement already satisfied: typing-extensions>=4.12 in
/usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (4.15.0)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-
packages (from sqlalchemy>=1.4.2->optuna) (3.2.4)
Requirement already satisfied: MarkupSafe>=0.9.2 in
/usr/local/lib/python3.12/dist-packages (from Mako->alembic>=1.5.0->optuna)
(3.0.3)
```

[ ]:
```
import random
import math
import optuna
import time
import math
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

[ ]:
```
# n = num of cities, (low, high) = range
def generate_TSP(n, seed=0, low=0.0, high=100.0):
    rnd = random.Random(seed)
    coords = [(rnd.uniform(low, high), rnd.uniform(low, high)) for _ in␣
 ↪range(n)]
    return coords
```

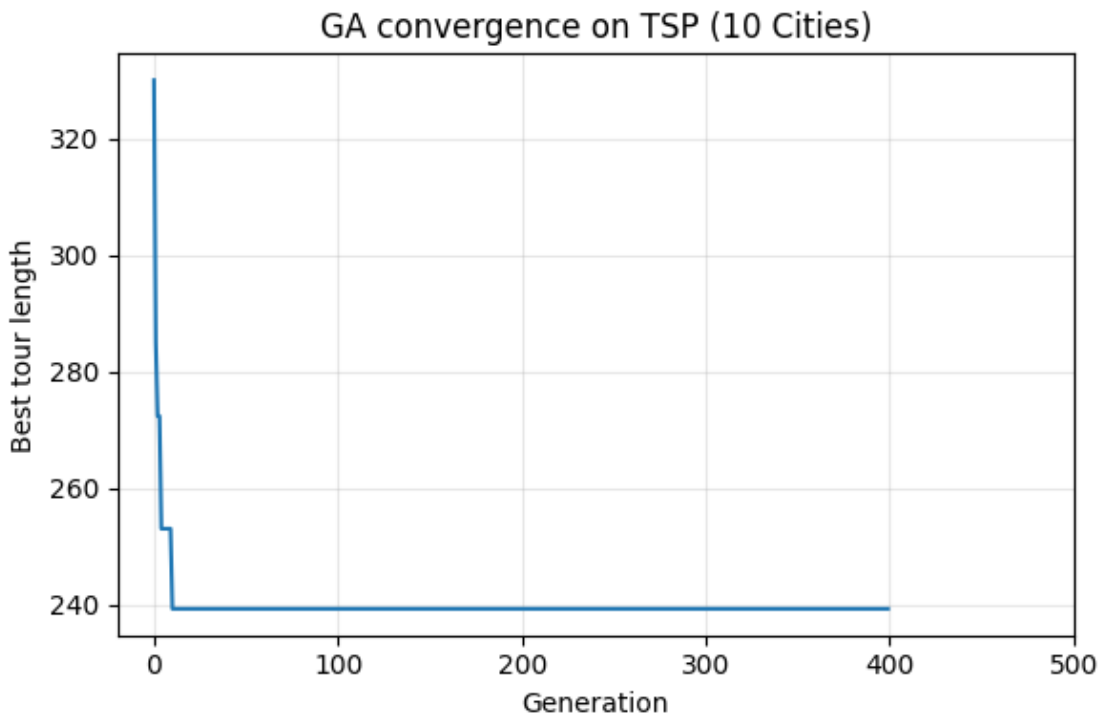
```
    print("GA runtime  :", elapsed_GA)
    print("Optimized GA best length :", opt_best_len_ga)
    print("Optimized GA best tour   :", opt_best_tour_ga)
    print("Optimized GA runtime  :", opt_elapsed_GA)
    print("ACO best length:", best_len_aco)
    print("ACO best tour  :", best_tour_aco)
    print("ACO runtime  :", elapsed_ACO)
    print("Optimized ACO best length :", opt_best_len_aco)
    print("Optimized ACO best tour   :", opt_best_tour_aco)
    print("Optimized ACO runtime  :", opt_elapsed_ACO)
    print()

    return (coords, best_tour_ga, best_len_ga, ga_best_costs, elapsed_GA,␣
↪best_tour_aco, best_len_aco, aco_best_costs, elapsed_ACO, opt_best_tour_ga,␣
↪opt_best_len_ga, opt_ga_best_costs, opt_elapsed_GA, opt_best_tour_aco,␣
↪opt_best_len_aco, opt_aco_best_costs, opt_elapsed_ACO)
```

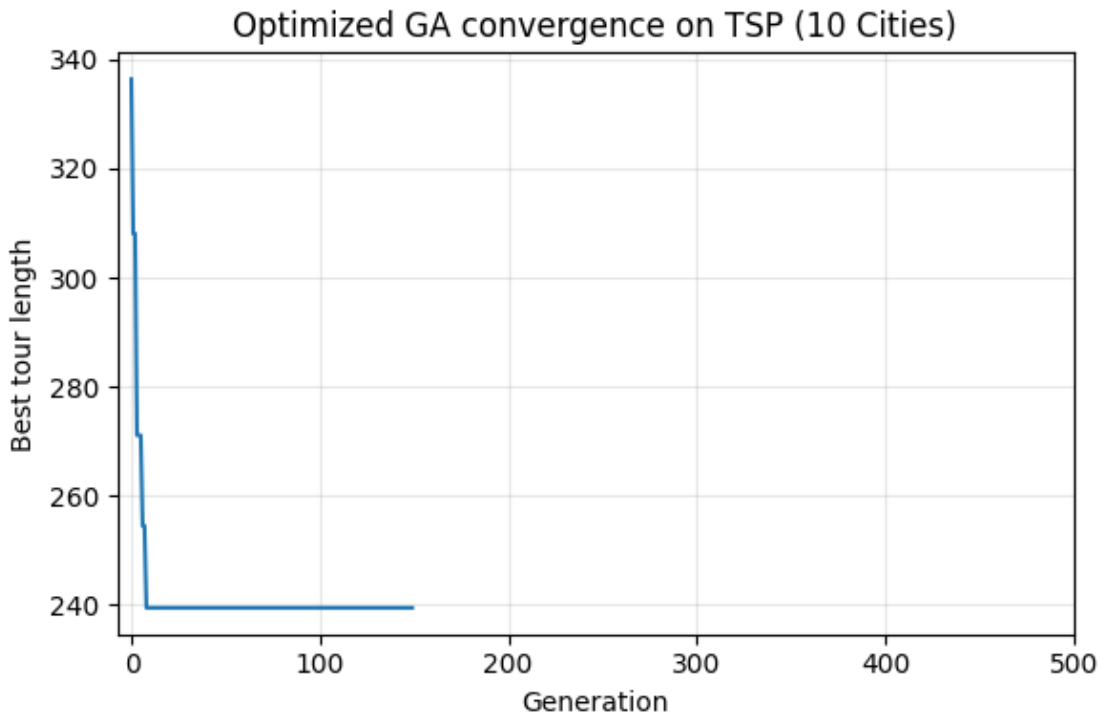
[ ]:
```
def plot_best_tour(coords, tour, title="Best GA TSP tour"):

    # Build x, y lists following the tour order and return to start
    xs = [coords[i][0] for i in tour] + [coords[tour[0]][0]]
    ys = [coords[i][1] for i in tour] + [coords[tour[0]][1]]

    plt.figure(figsize=(6, 6))
    plt.plot(xs, ys, marker='o')

    # Label each city with its index
    for idx in tour:
        x, y = coords[idx]
        plt.text(x, y, str(idx), fontsize=9, ha='right', va='bottom')

    plt.title(title)
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.grid(True, alpha=0.3)
    plt.gca().set_aspect('equal', 'box')  # keep aspect ratio
    plt.tight_layout()
    plt.show()
```

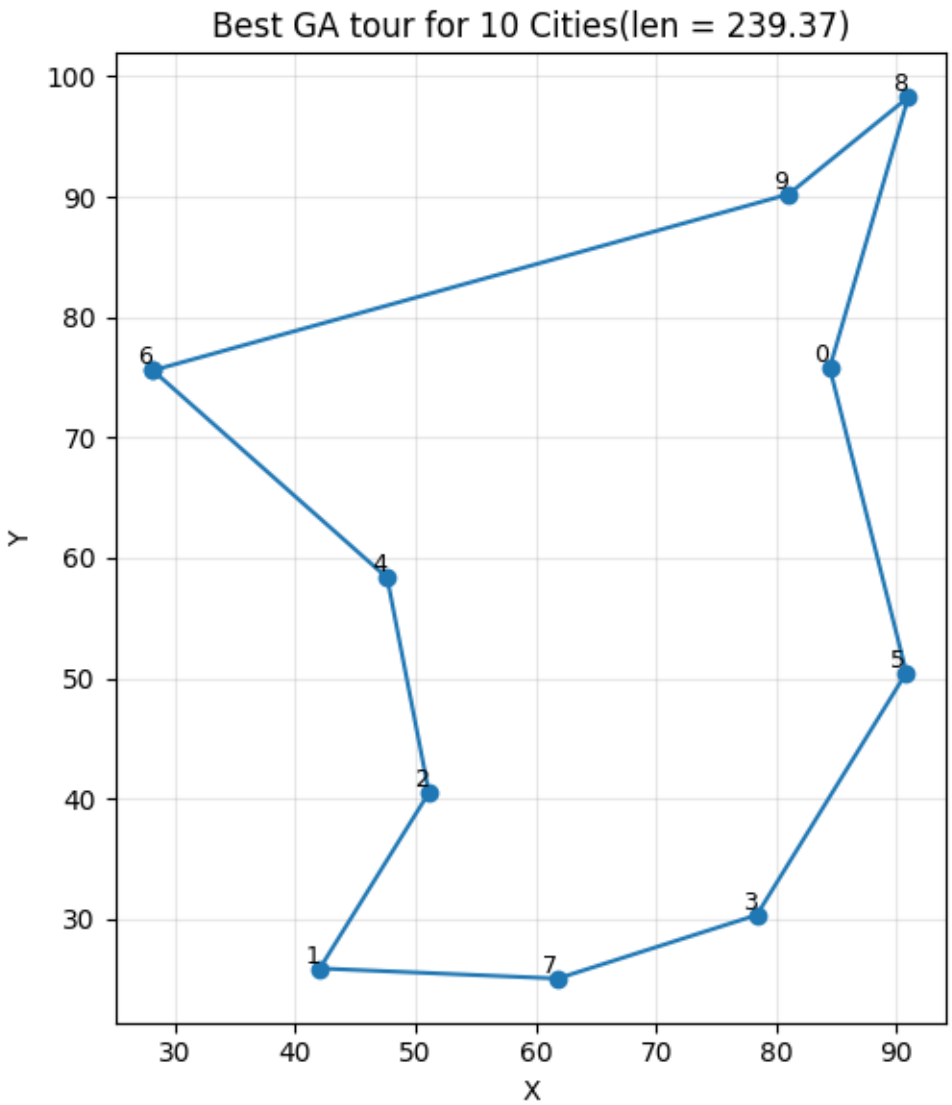
Execution of GA(Optimized and Original) and ACO(Optimized and Original) for TSP with 10 Cities.

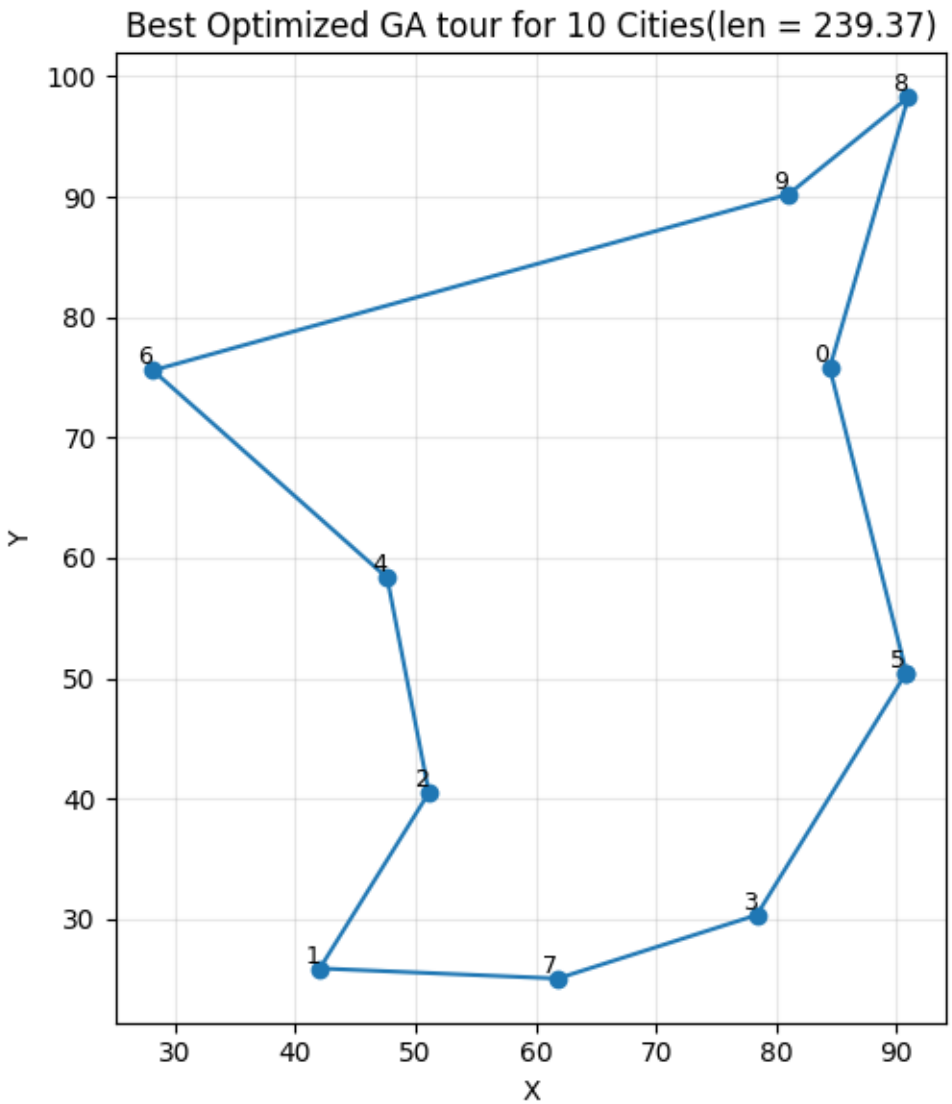
[ ]:
```
#for 10 cities
```

```
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_10, best_tour_ga_10, title=f"Best GA tour for 10␣
↪Cities(len = {best_len_ga_10:.2f})")
plot_best_tour(coords_10, opt_best_tour_ga_10, title=f"Best Optimized GA tour␣
↪for 10 Cities(len = {opt_best_len_ga_10:.2f})")
```




Optimized GA convergence on TSP (10 Cities)
Best tour length
Generation
340
320
300
280
260
240
0
100
200
300
400
500


Best GA tour for 10 Cities(len = 239.37)
100
90
80
70
60
50
40
30
Y
30
40
50
60
70
80
90
X
8
9
6
0
4
5
2
3
1
7

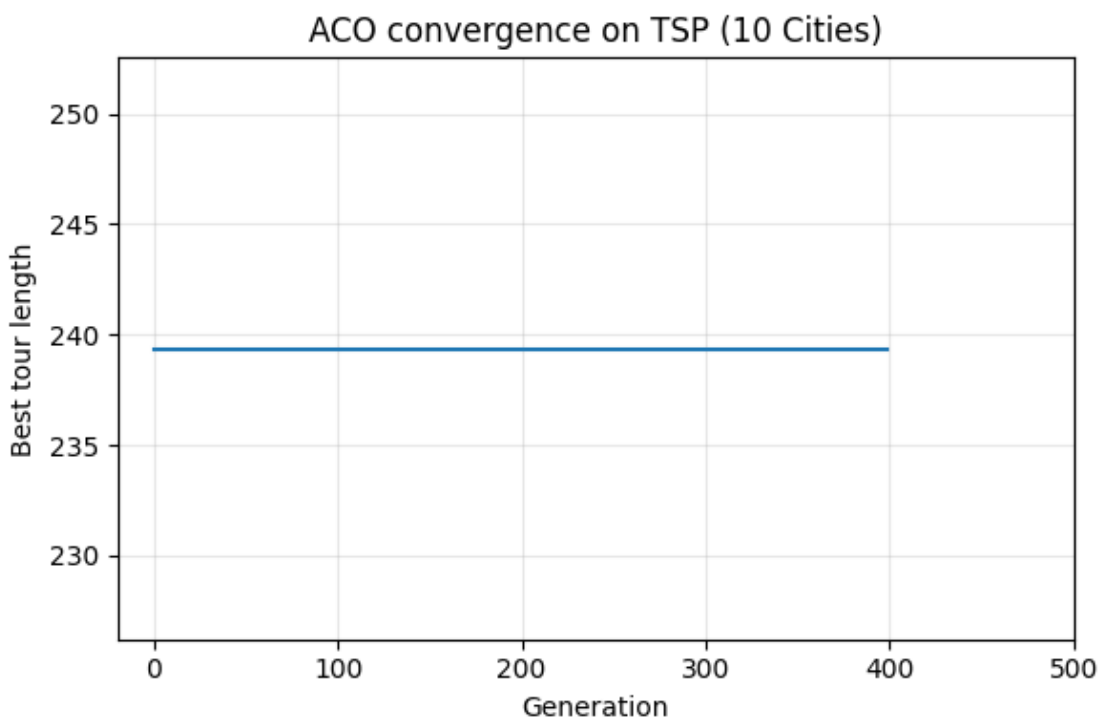


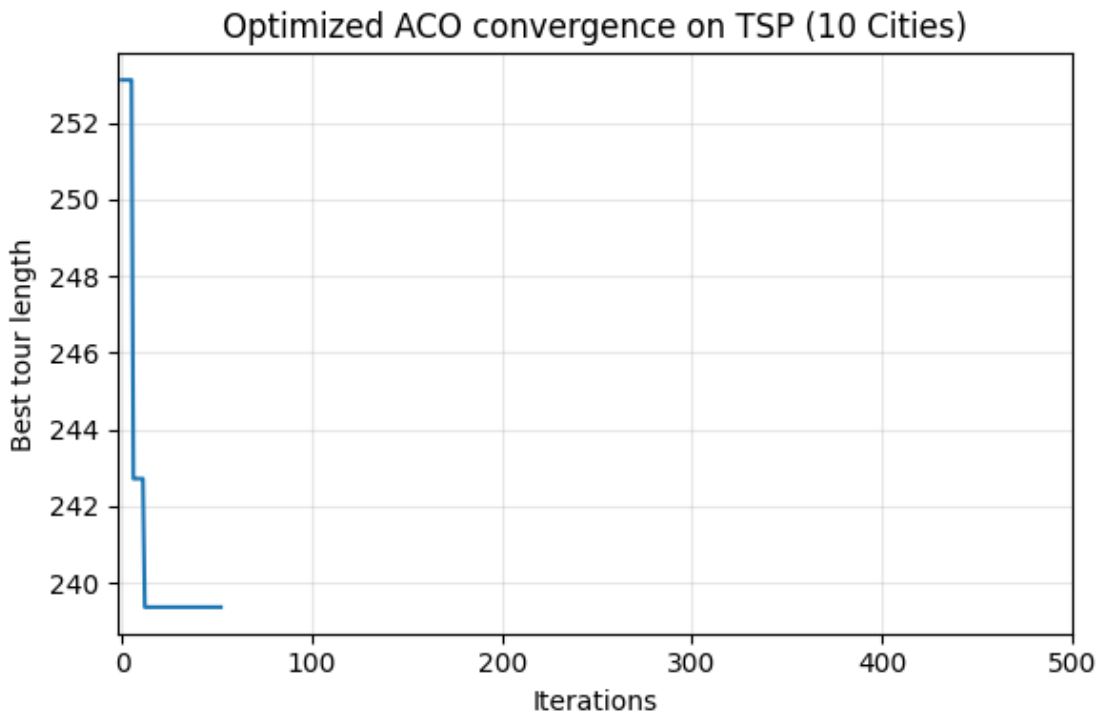
When the number of Cities in the TSP problem is 10 cities, both the original GA algorithm and the Optimized version of GA algorithm has achieved the same cost which seems to be the optimal value. According to the result, the tour of both the versions are the same.

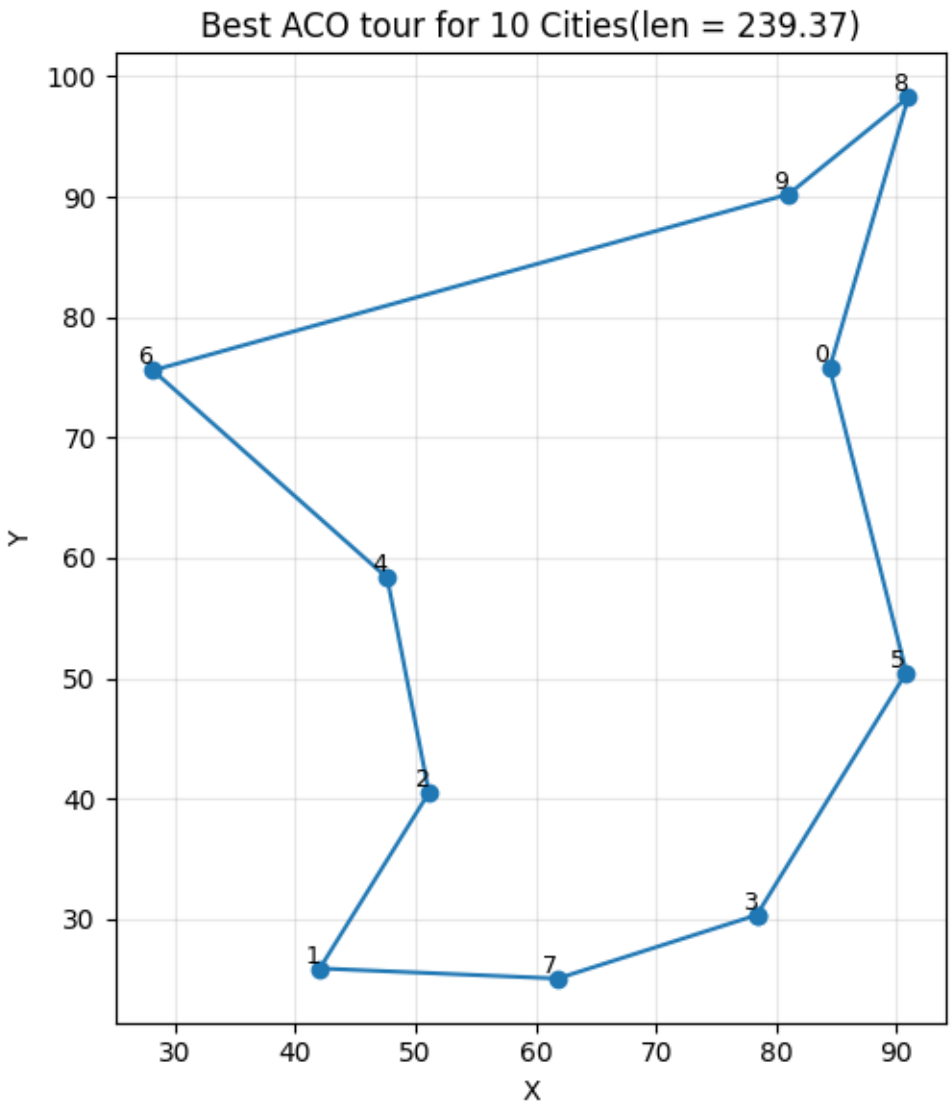
According to the convergence Graphs it can be seen that the Optimized version has reduced the number of iterations to reach the final solution which causes an increase in efficiency after optimization.

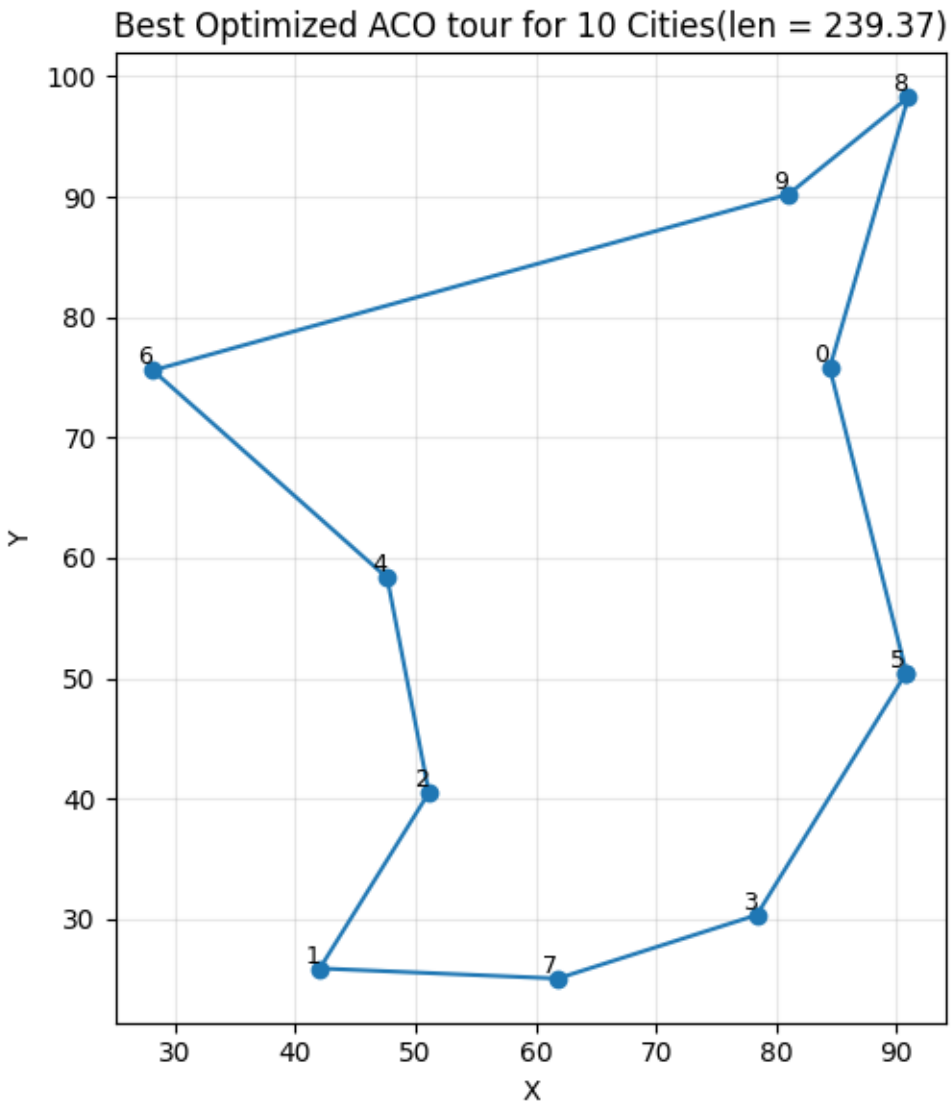
[ ]:
```
plt.figure(figsize=(6,4))
plt.plot(aco_best_costs_10)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("ACO convergence on TSP (10 Cities)")
plt.xlim(xmax=500)
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(opt_aco_best_costs_10)
plt.xlabel("Iterations")
plt.ylabel("Best tour length")
plt.title("Optimized ACO convergence on TSP (10 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_10, best_tour_aco_10, title=f"Best ACO tour for 10␣
↪Cities(len = {best_len_aco_10:.2f})")
plot_best_tour(coords_10, opt_best_tour_aco_10, title=f"Best Optimized ACO tour␣
↪for 10 Cities(len = {opt_best_len_aco_10:.2f})")
```




Optimized ACO convergence on TSP (10 Cities)
Best tour length
252
250
248
246
244
242
240
0
100
200
300
400
500
Iterations

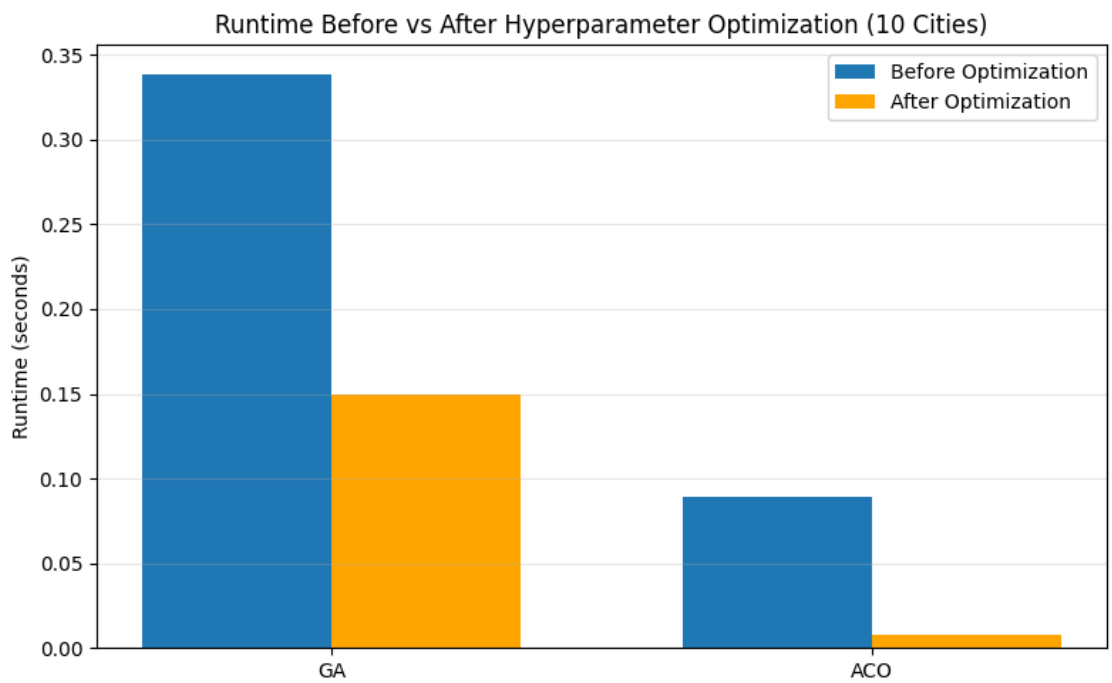

Best ACO tour for 10 Cities(len = 239.37)
100
90
80
70
60
50
40
30
Y
30
40
50
60
70
80
90
X
8
9
6
0
4
5
2
3
1
7



According to the Convergence graphs, it can be seen that the ACO is easily solving the TSP problem with 10 cities even without optimization(with respect to the cost). However, to increase the efficiency, optimization has been done.

[ ]:
```
algos = ["GA", "ACO"]
before = [elapsed_GA_10, elapsed_ACO_10]
after  = [opt_elapsed_GA_10, opt_elapsed_ACO_10]

x = np.arange(len(algos))
w = 0.35

plt.figure(figsize=(8,5))
```

```
plt.bar(x - w/2, before, width=w, label="Before Optimization")
plt.bar(x + w/2, after,  width=w, label="After Optimization", color="orange")

plt.xticks(x, algos)
plt.ylabel("Runtime (seconds)")
plt.title("Runtime Before vs After Hyperparameter Optimization (10 Cities)")
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



For TSP with 10 cities, both the original and Optimized versions of GA and ACO reach the similar final cost for travelling. However, after optimization, the runtime efficiency of both GA and ACO has improved significantly.

Execution of GA(Optimized and Original) and ACO(Optimized and Original) for TSP with 50 Cities.

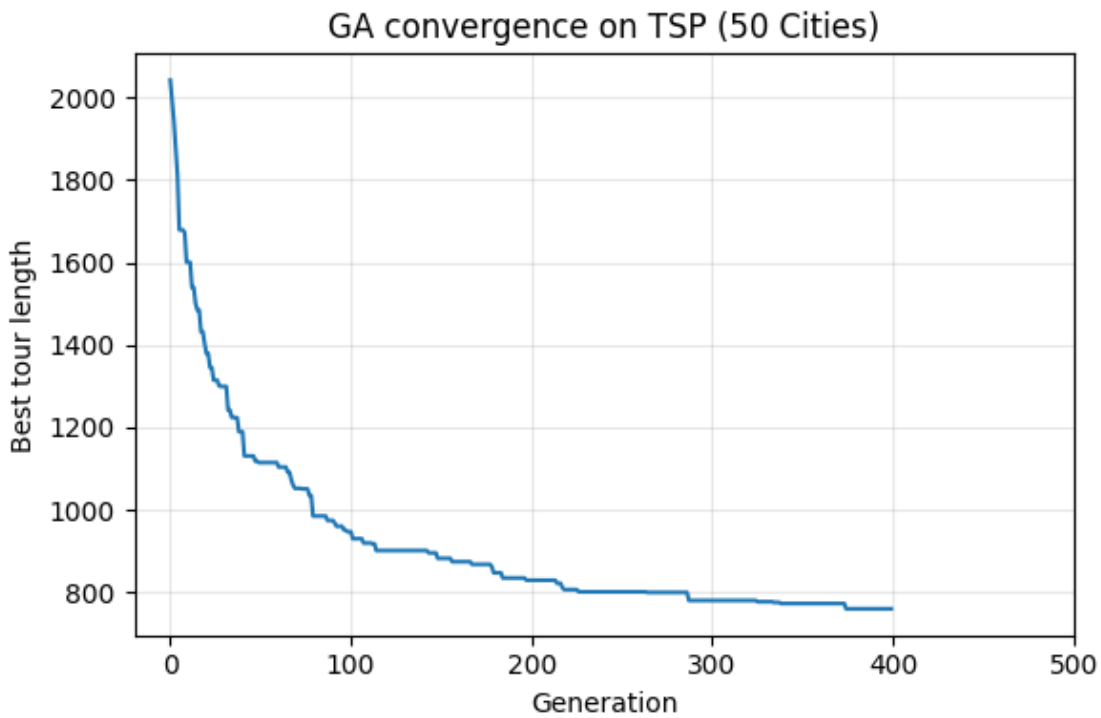
[ ]:
```
#for 50 cities
coords_50, best_tour_ga_50, best_len_ga_50, ga_best_costs_50, elapsed_GA_50,␣
↪best_tour_aco_50, best_len_aco_50, aco_best_costs_50, elapsed_ACO_50,␣
↪opt_best_tour_ga_50, opt_best_len_ga_50, opt_ga_best_costs_50,␣
↪opt_elapsed_GA_50, opt_best_tour_aco_50, opt_best_len_aco_50,␣
↪opt_aco_best_costs_50, opt_elapsed_ACO_50=execute(50)
```

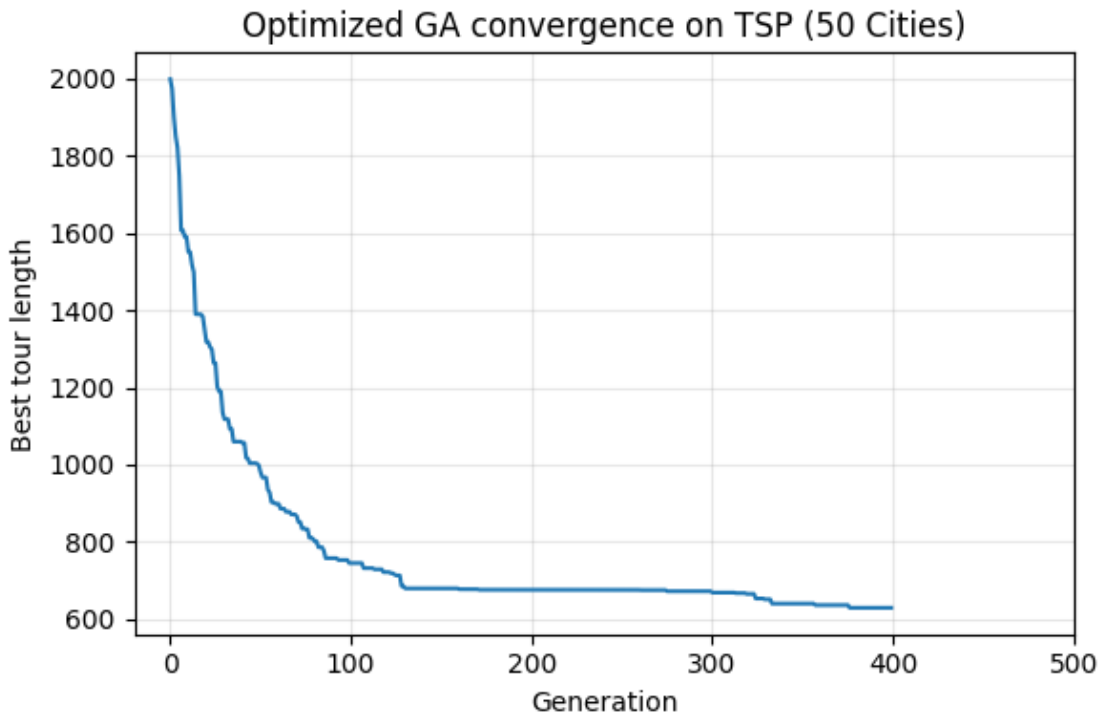
```
=== n = 50 ===
```

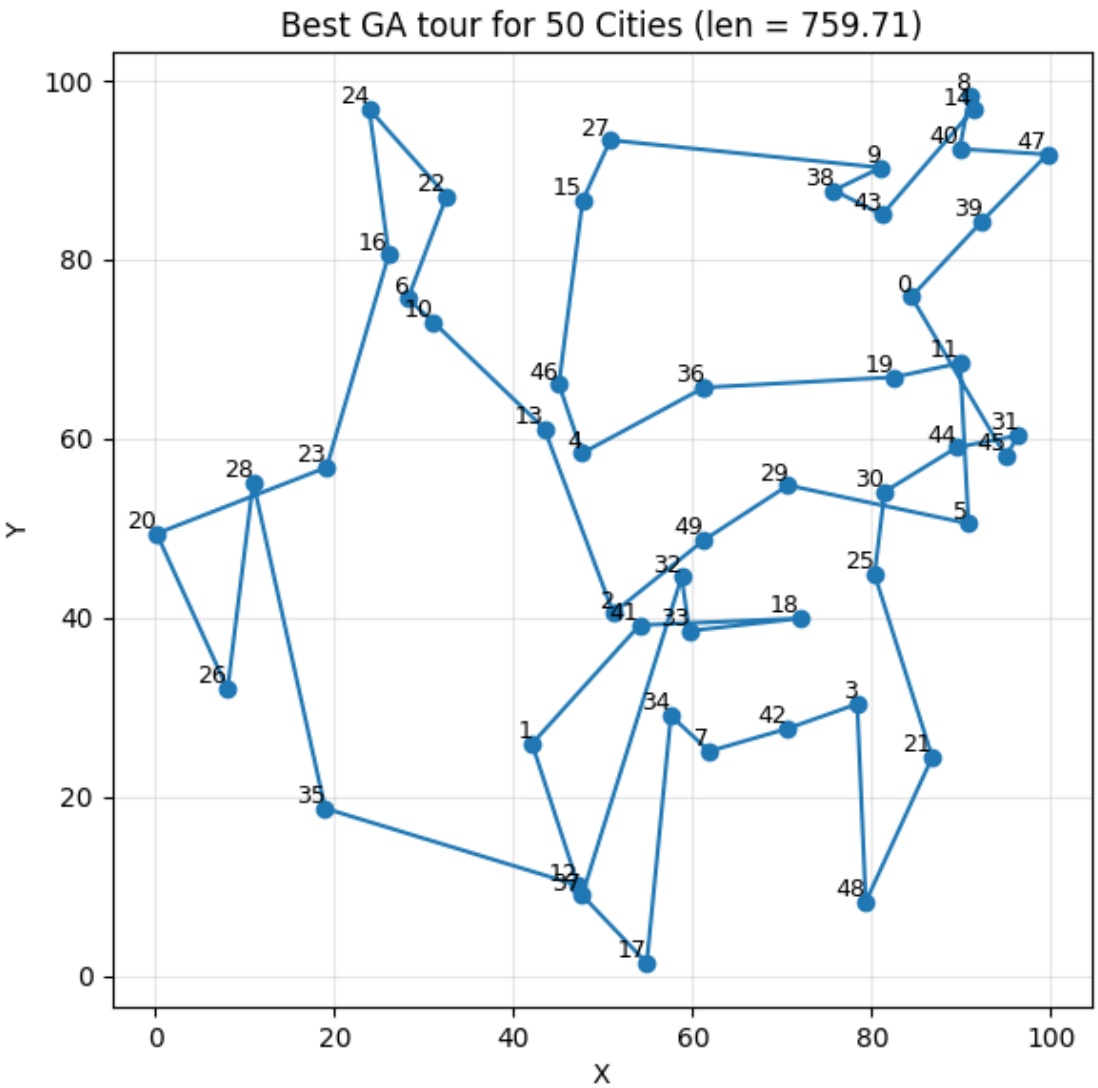
```
[I 2025-11-23 18:03:37,960] Trial 118 finished with value: 1000545.7389275517
and parameters: {'n_ants': 87, 'alpha': 1.847229380668994, 'beta':
8.511239358622298, 'rho': 1, 'Q': 656, 'iterations': 180}. Best is trial 11 with
value: 534.4944474546.
[I 2025-11-23 18:04:08,764] Trial 119 finished with value: 1000535.6689653026
and parameters: {'n_ants': 67, 'alpha': 0.5809310166563884, 'beta':
8.946542919918791, 'rho': 1, 'Q': 719, 'iterations': 333}. Best is trial 11 with
value: 534.4944474546.

Best Params of ACO {'n_ants': 97, 'alpha': 0.5280794565476233, 'beta':
5.644131714762152, 'rho': 1, 'Q': 17, 'iterations': 400}
GA best length : 759.7132568728946
GA best tour   : [36, 4, 46, 15, 27, 9, 38, 43, 14, 8, 40, 47, 39, 0, 45, 31,
44, 30, 25, 21, 48, 3, 42, 7, 34, 17, 37, 32, 33, 18, 41, 1, 12, 35, 28, 26, 20,
23, 16, 24, 22, 6, 10, 13, 2, 49, 29, 5, 11, 19]
GA runtime  : 1.2590982139990956
Optimized GA best length : 628.6860936500279
Optimized GA best tour   : [27, 24, 16, 10, 46, 13, 4, 36, 29, 30, 5, 45, 31, 0,
43, 38, 9, 8, 14, 47, 40, 39, 11, 19, 44, 25, 33, 2, 41, 32, 49, 18, 42, 3, 21,
48, 17, 12, 37, 7, 34, 1, 26, 35, 20, 28, 23, 6, 22, 15]
Optimized GA runtime  : 4.834442715000478
ACO best length: 536.2562242266749
ACO best tour  : [49, 32, 33, 41, 2, 34, 7, 42, 3, 21, 48, 17, 37, 12, 1, 35,
26, 20, 28, 23, 10, 6, 16, 22, 24, 15, 27, 38, 43, 9, 40, 14, 8, 47, 39, 0, 19,
11, 44, 45, 31, 5, 30, 25, 18, 29, 36, 46, 13, 4]
ACO runtime  : 9.420383791000859
Optimized ACO best length : 534.4944474546
Optimized ACO best tour   : [49, 32, 33, 41, 2, 34, 7, 42, 3, 21, 48, 17, 37,
12, 1, 35, 26, 20, 28, 23, 10, 6, 16, 22, 24, 15, 27, 38, 43, 9, 40, 14, 8, 47,
39, 0, 19, 11, 31, 45, 44, 5, 30, 25, 18, 29, 36, 46, 13, 4]
Optimized ACO runtime  : 17.10560003700084
```

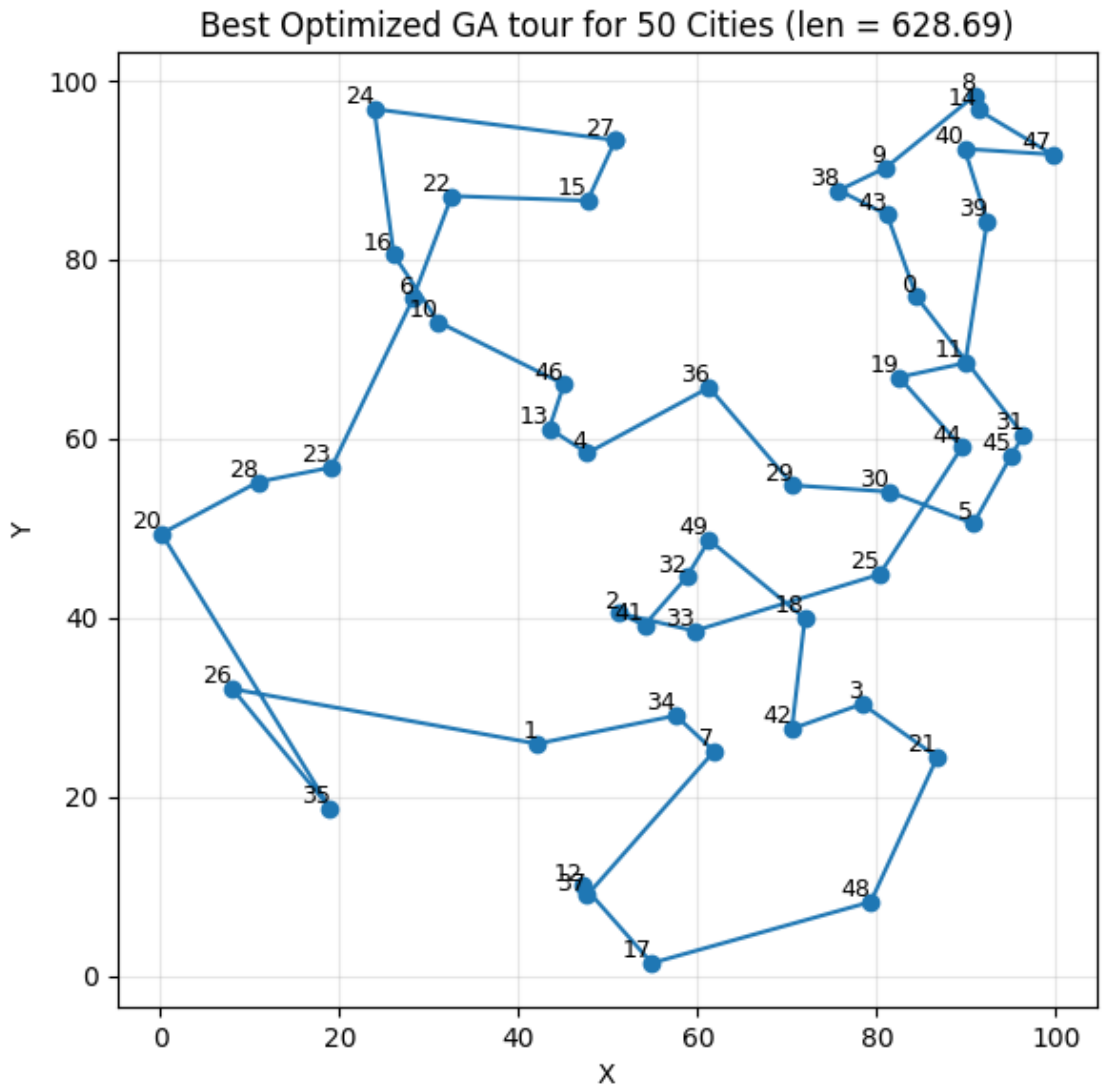
[ ]:
```python
plt.figure(figsize=(6,4))
plt.plot(ga_best_costs_50)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("GA convergence on TSP (50 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(opt_ga_best_costs_50)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("Optimized GA convergence on TSP (50 Cities)")
```

```
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_50, best_tour_ga_50, title=f"Best GA tour for 50 Cities␣
↪(len = {best_len_ga_50:.2f})")
plot_best_tour(coords_50, opt_best_tour_ga_50, title=f"Best Optimized GA tour␣
↪for 50 Cities (len = {opt_best_len_ga_50:.2f})")
```




Optimized GA convergence on TSP (50 Cities)
Best tour length
2000
1800
1600
1400
1200
1000
800
600
0
100
200
300
400
500
Generation

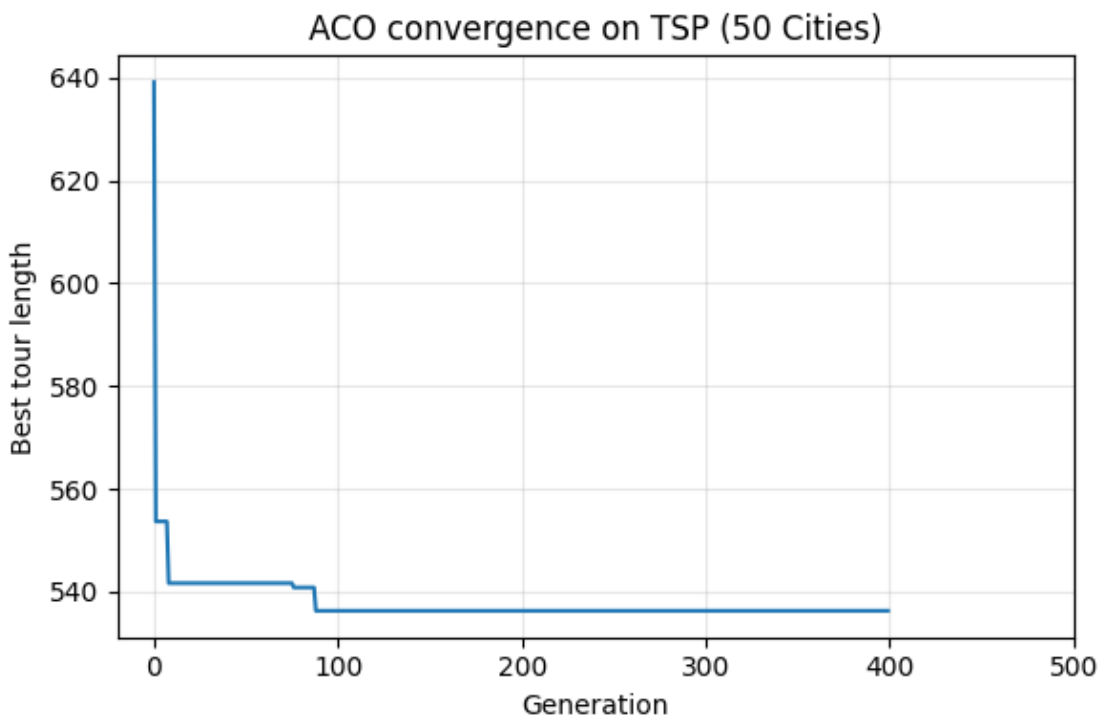

Best GA tour for 50 Cities (len = 759.71)
X
Y
0
20
40
60
80
100
24
8
14
27
40
47
9
38
43
22
15
39
16
6
10
0
11
46
36
19
13
4
31
44
45
28
23
29
30
20
5
49
32
25
2
41
33
18
26
34
3
42
1
7
21
35
12
37
48
17

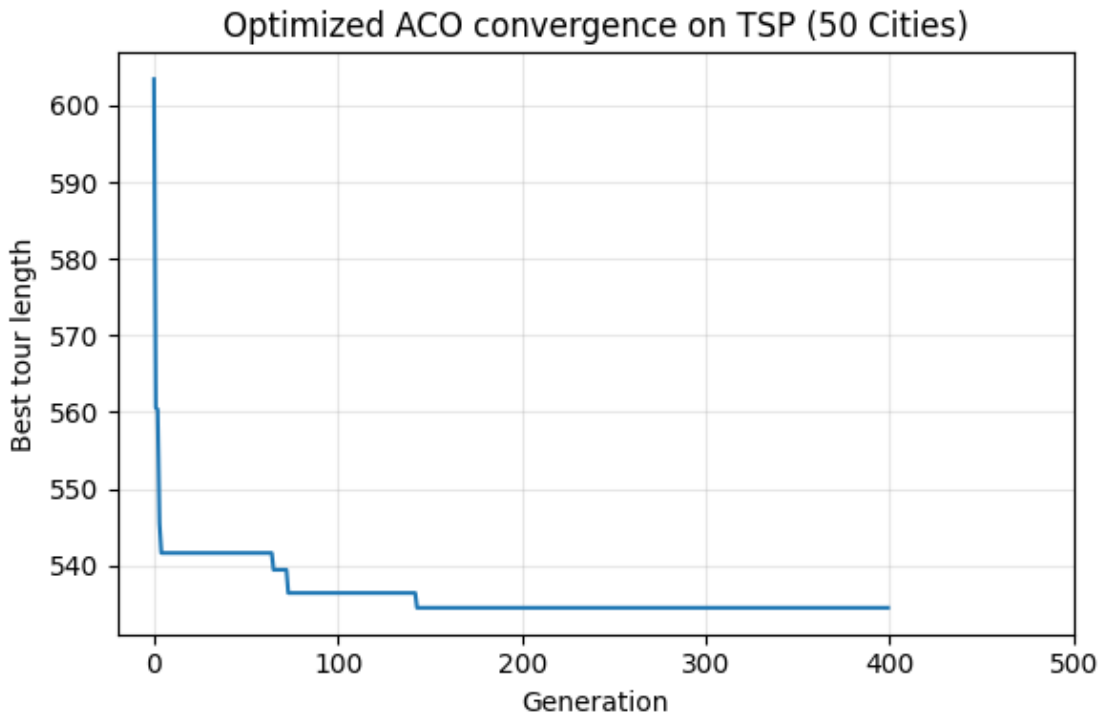


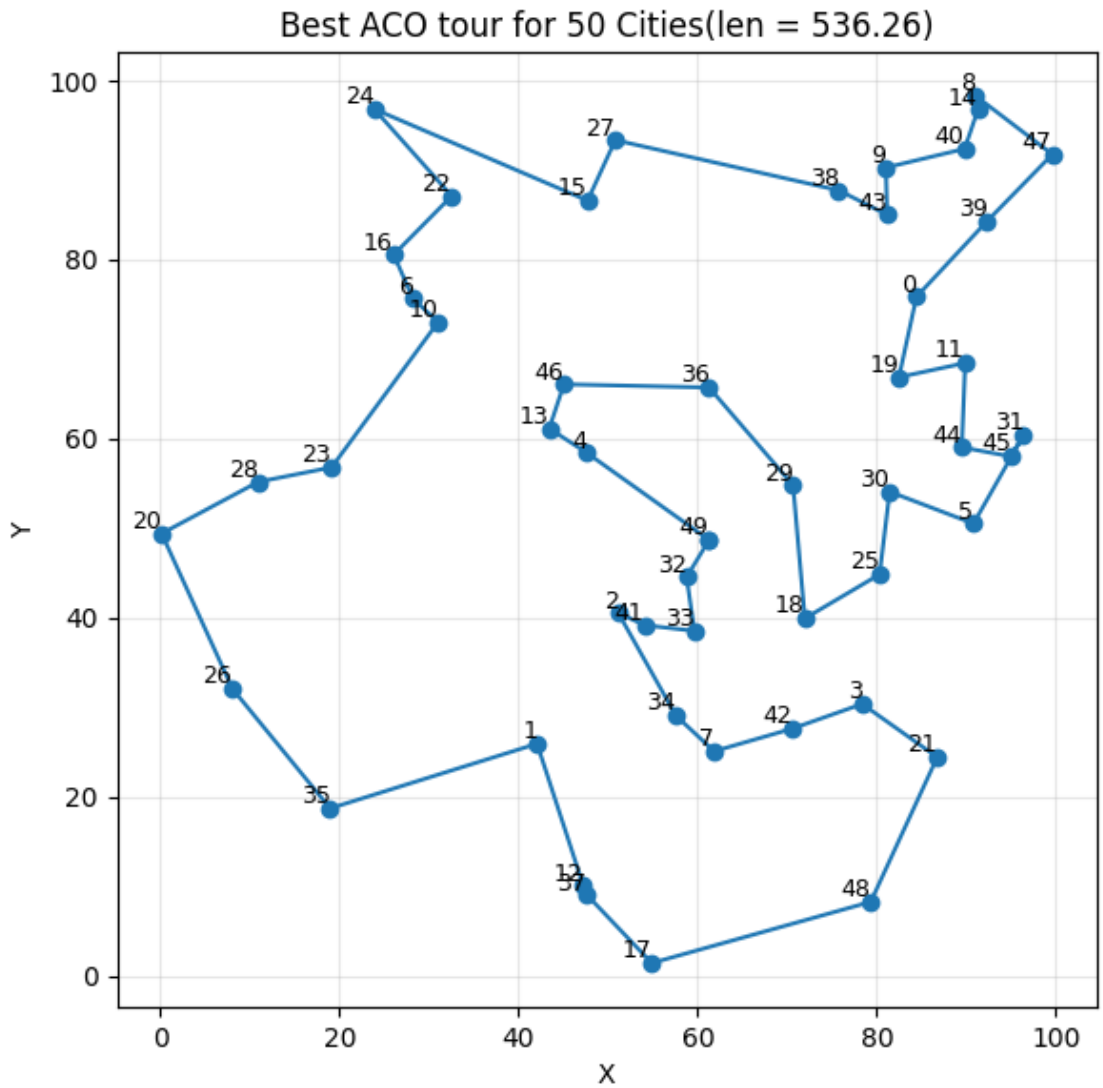
According to the Convergence Graphs above, the Optimized version of GA converges to better solutions in earlier iterations compare to the original GA. Moreover, the Optimized version has reached a better solution within the iteration limit. According to the graphs representing the tour of GA, the optimized version has found a less complex path which reflects the quality of the output after optimization.

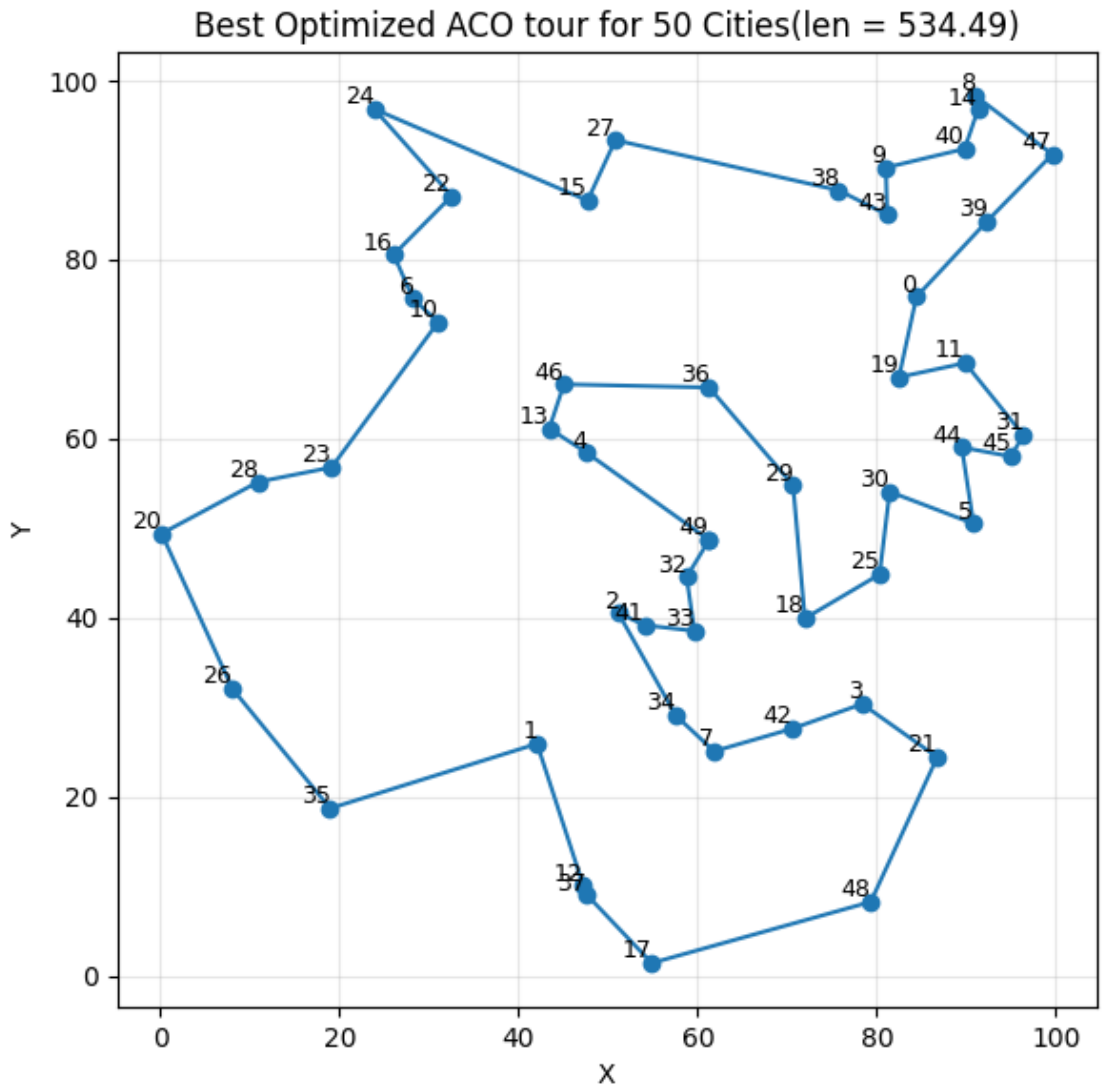
[ ]:
```
plt.figure(figsize=(6,4))
plt.plot(aco_best_costs_50)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("ACO convergence on TSP (50 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(opt_aco_best_costs_50)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("Optimized ACO convergence on TSP (50 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_50, best_tour_aco_50, title=f"Best ACO tour for 50␣
↪Cities(len = {best_len_aco_50:.2f})")
plot_best_tour(coords_50, opt_best_tour_aco_50, title=f"Best Optimized ACO tour␣
↪for 50 Cities(len = {opt_best_len_aco_50:.2f})")
```




Optimized ACO convergence on TSP (50 Cities)
Best tour length
600
590
580
570
560
550
540
0
100
200
300
400
500
Generation


Best ACO tour for 50 Cities(len = 536.26)
X
Y
0
20
40
60
80
100
24
27
8
14
40
47
9
38
43
39
22
15
16
6
10
0
11
19
46
36
13
4
31
44
45
28
23
29
30
5
20
49
32
25
2
41
33
18
26
34
42
3
7
21
1
35
12
37
48
17



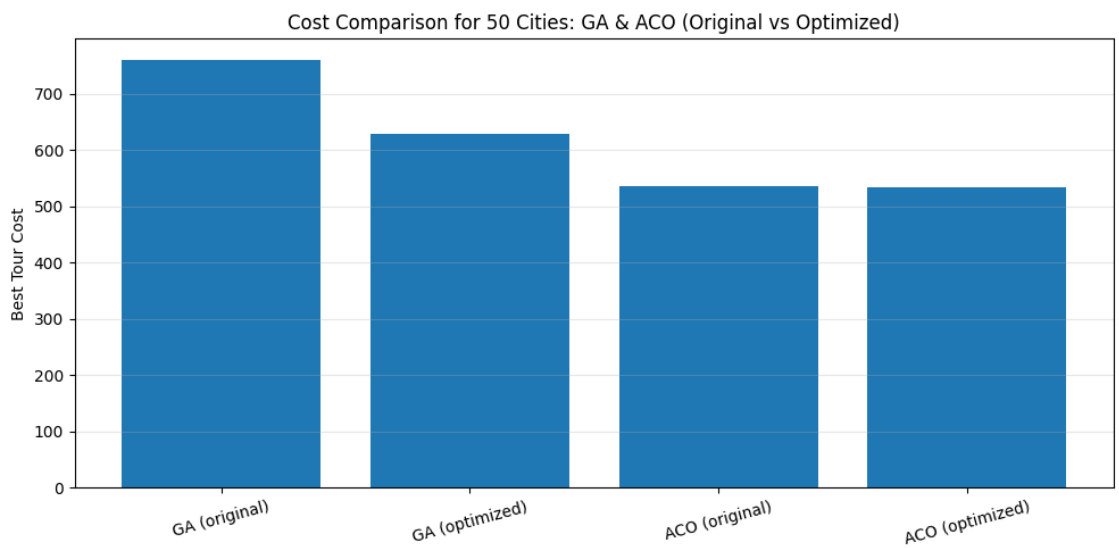
By refering the Convergence graphs above, it can be seen that Optimized ACO converges to better solutions slightly faster than the original ACO. Finally, the Optimized ACO reached a better solution than the Original ACO within the iteration limit. However, there is no drastical improvement in the cost as only two edges are changed in the optimized version.

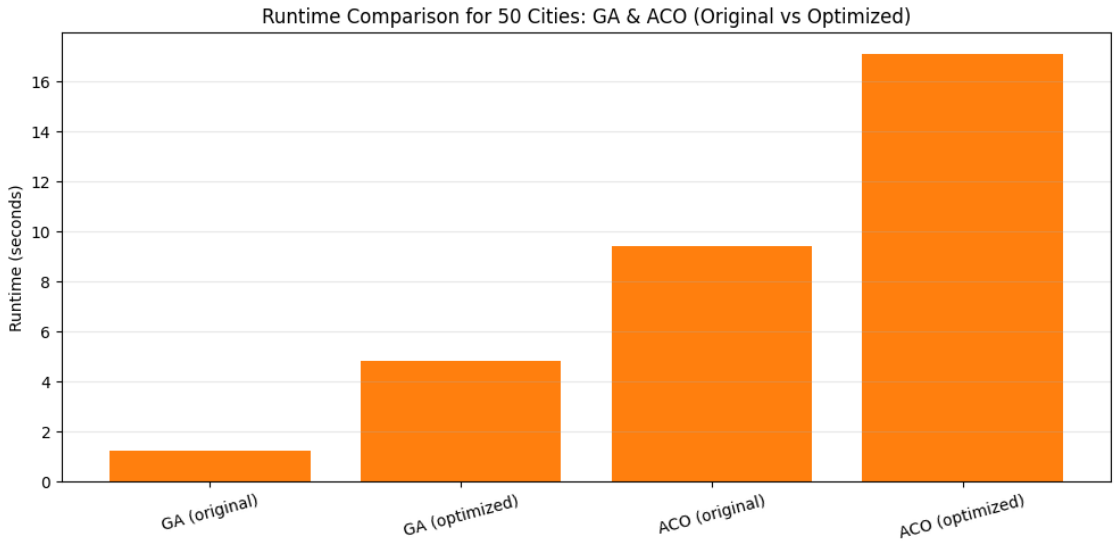
[ ]:
```
data = {
    "Algorithm": [
        "GA (Original)", "GA (Optimized)",
        "ACO (Original)", "ACO (Optimized)"
    ],
    "Best Cost": [
        best_len_ga_50, opt_best_len_ga_50,
        best_len_aco_50, opt_best_len_aco_50
```

```
    ],
    "Runtime (sec)": [
        elapsed_GA_50, opt_elapsed_GA_50,
        elapsed_ACO_50, opt_elapsed_ACO_50
    ]
}

df = pd.DataFrame(data)

display(df)


# Labels for x-axis
labels = [
    "GA (original)",
    "GA (optimized)",
    "ACO (original)",
    "ACO (optimized)",
]

# Corresponding cost + time arrays
costs = [
    best_len_ga_50,
    opt_best_len_ga_50,
    best_len_aco_50,
    opt_best_len_aco_50,
]

times = [
    elapsed_GA_50,
    opt_elapsed_GA_50,
    elapsed_ACO_50,
    opt_elapsed_ACO_50,
]

x = np.arange(len(labels))  # x positions



#   PLOT 1 - COST Comparison
plt.figure(figsize=(10,5))
plt.bar(x, costs, color="tab:blue")
plt.xticks(x, labels, rotation=15)
plt.ylabel("Best Tour Cost")
plt.title("Cost Comparison for 50 Cities: GA & ACO (Original vs Optimized)")
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
```

```
plt.show()


#   PLOT 2 - TIME Comparison
plt.figure(figsize=(10,5))
plt.bar(x, times, color="tab:orange")
plt.xticks(x, labels, rotation=15)
plt.ylabel("Runtime (seconds)")
plt.title("Runtime Comparison for 50 Cities: GA & ACO (Original vs Optimized)")
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
         Algorithm   Best Cost  Runtime (sec)
0    GA (Original)  759.713257       1.259098
1   GA (Optimized)  628.686094       4.834443
2   ACO (Original)  536.256224       9.420384
3  ACO (Optimized)  534.494447      17.105600
```





From the above bar graph related to Cost(Tour length) comparison, it can be seen that both GA and ACO has improved it's performance after Hyper Parameter Optimization(HPO). However, the runtime of both GA and ACO has increased after optimization in order to achieve the improvement.

Optimized GA has achieved a considerable level of improvement, which is worth to invest more time. However, ACO has achieved only a minor improvement by investing considerable amount of time. Therefore, It can be considered not worthwhile to invest time.

Execution of GA(Optimized and Original) and ACO(Optimized and Original) for TSP with 100 Cities.

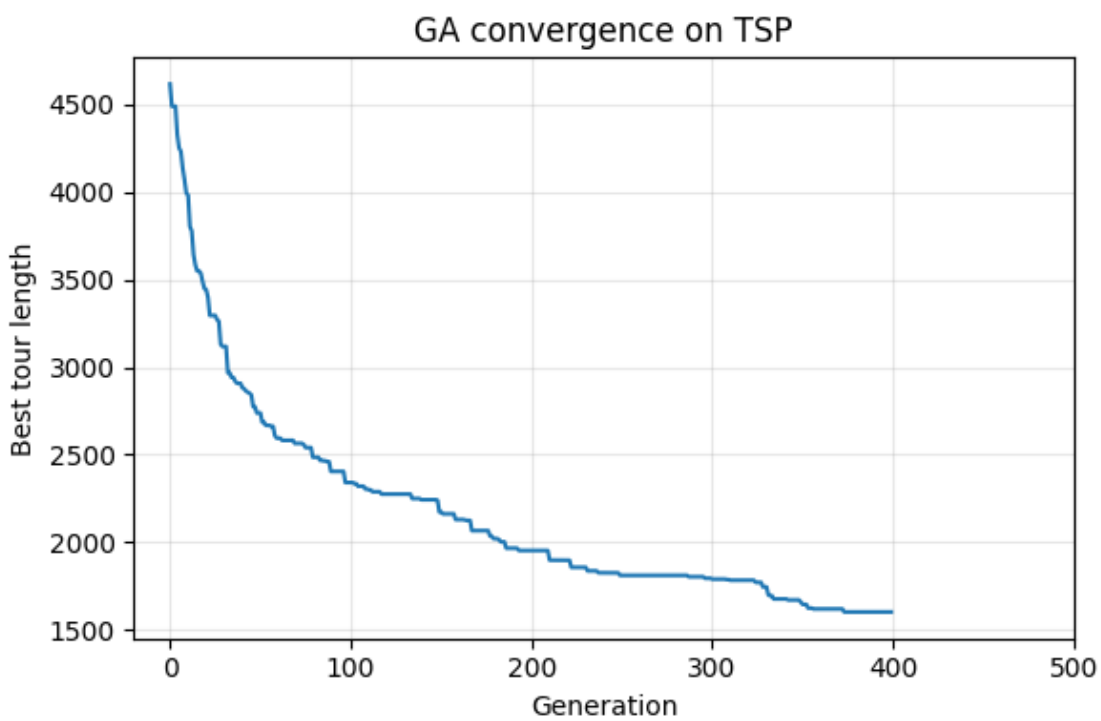
[ ]:
```
#for 100 cities
coords_100, best_tour_ga_100, best_len_ga_100, ga_best_costs_100,␣
↪elapsed_GA_100, best_tour_aco_100, best_len_aco_100, aco_best_costs_100,␣
↪elapsed_ACO_100, opt_best_tour_ga_100, opt_best_len_ga_100,␣
↪opt_ga_best_costs_100, opt_elapsed_GA_100, opt_best_tour_aco_100,␣
↪opt_best_len_aco_100, opt_aco_best_costs_100,␣
↪opt_elapsed_ACO_100=execute(100)
```

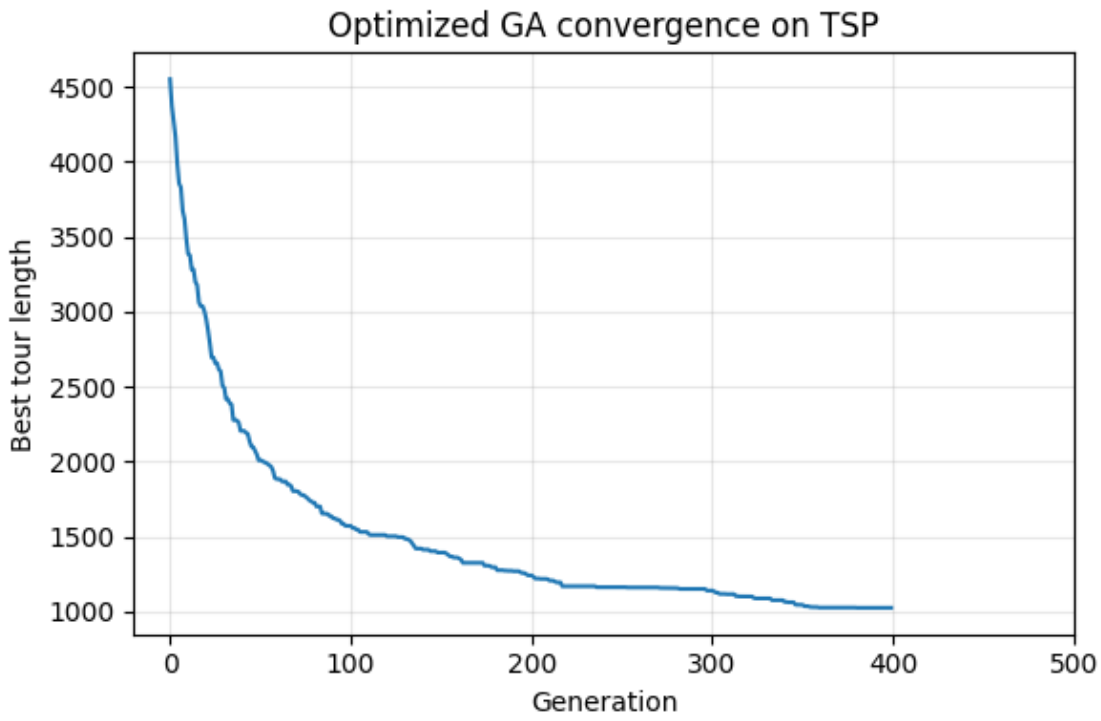
```
=== n = 100 ===

[I 2025-11-23 18:05:45,371] A new study created in memory with name: no-
name-907fbcf6-ab50-4294-b6d2-0f5230fcd395
[I 2025-11-23 18:05:48,978] Trial 0 finished with value: 1735.0757516701763 and
parameters: {'population_size': 25, 'crossover_rate': 0.9119675168960458,
'mutation_rate': 0.012189694614178044, 'elitism': 2, 'tournament_k': 10,
'generations': 400}. Best is trial 0 with value: 1735.0757516701763.
[I 2025-11-23 18:06:02,803] Trial 1 finished with value: 1317.4025533479332 and
parameters: {'population_size': 99, 'crossover_rate': 0.8004481854639751,
'mutation_rate': 0.0015082710453590447, 'elitism': 2, 'tournament_k': 6,
'generations': 400}. Best is trial 1 with value: 1317.4025533479332.
```

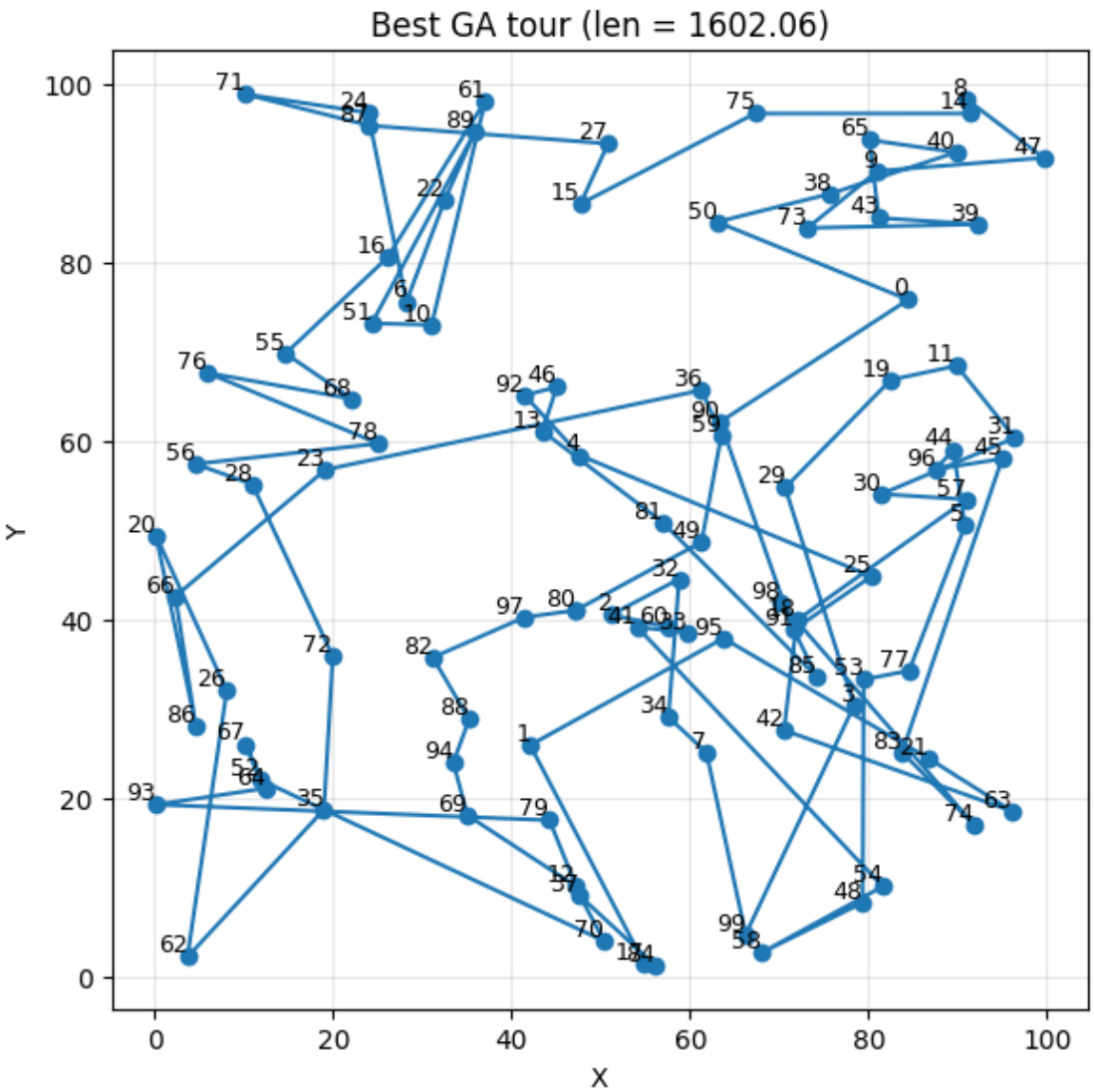
```
2.19948055015441, 'rho': 0, 'Q': 558, 'iterations': 400}
GA best length : 1602.0591228932597
GA best tour   : [80, 49, 59, 36, 23, 66, 86, 20, 26, 62, 35, 72, 28, 56, 78,
76, 68, 55, 16, 61, 10, 51, 89, 22, 6, 24, 71, 87, 27, 15, 75, 14, 8, 47, 9, 73,
39, 43, 65, 40, 38, 50, 0, 90, 91, 25, 4, 92, 46, 13, 81, 85, 98, 74, 83, 45,
96, 44, 5, 77, 53, 48, 58, 54, 41, 33, 60, 2, 32, 34, 7, 99, 3, 29, 19, 11, 31,
30, 57, 18, 42, 63, 21, 95, 1, 17, 84, 37, 70, 52, 67, 64, 93, 79, 12, 69, 94,
88, 82, 97]
GA runtime  : 4.913046868998208
Optimized GA best length : 1025.6092656157416
Optimized GA best tour   : [72, 78, 68, 23, 56, 76, 20, 66, 28, 55, 51, 10, 6,
16, 13, 46, 92, 4, 59, 90, 36, 81, 32, 41, 60, 80, 2, 97, 82, 88, 1, 79, 70, 7,
34, 42, 85, 95, 33, 49, 98, 30, 29, 96, 44, 39, 47, 40, 14, 8, 65, 9, 38, 75,
50, 15, 22, 87, 24, 71, 61, 89, 27, 73, 43, 0, 19, 11, 31, 45, 5, 57, 25, 18,
91, 21, 83, 3, 77, 53, 74, 63, 48, 54, 99, 58, 17, 84, 37, 12, 69, 94, 35, 67,
26, 86, 93, 62, 64, 52]
Optimized GA runtime  : 21.69103116900078
ACO best length: 797.4542888367382
ACO best tour  : [66, 20, 56, 28, 23, 78, 68, 55, 76, 51, 6, 10, 16, 22, 89, 61,
24, 87, 71, 15, 27, 75, 50, 73, 38, 43, 9, 65, 40, 14, 8, 47, 39, 0, 19, 11, 31,
45, 44, 96, 57, 5, 25, 30, 29, 59, 90, 36, 46, 92, 13, 4, 81, 49, 32, 60, 33,
95, 98, 18, 91, 85, 3, 53, 77, 83, 21, 74, 63, 54, 48, 58, 99, 84, 17, 70, 37,
12, 79, 69, 94, 88, 1, 34, 7, 42, 41, 2, 80, 97, 82, 72, 26, 86, 67, 52, 64, 35,
62, 93]
ACO runtime  : 72.60352543499903
Optimized ACO best length : 793.4150009758907
Optimized ACO best tour   : [81, 49, 32, 95, 33, 60, 41, 2, 80, 97, 82, 72, 26,
86, 67, 52, 64, 35, 62, 93, 66, 20, 56, 28, 23, 78, 68, 55, 76, 71, 87, 24, 61,
89, 22, 16, 6, 10, 51, 15, 27, 75, 50, 73, 38, 43, 9, 65, 40, 14, 8, 47, 39, 0,
11, 19, 30, 96, 44, 45, 31, 5, 57, 25, 77, 53, 3, 83, 21, 63, 74, 54, 48, 58,
99, 84, 17, 70, 37, 12, 79, 69, 94, 88, 1, 34, 7, 42, 85, 91, 18, 98, 29, 59,
90, 36, 46, 92, 13, 4]
Optimized ACO runtime  : 53.98003900099866
```

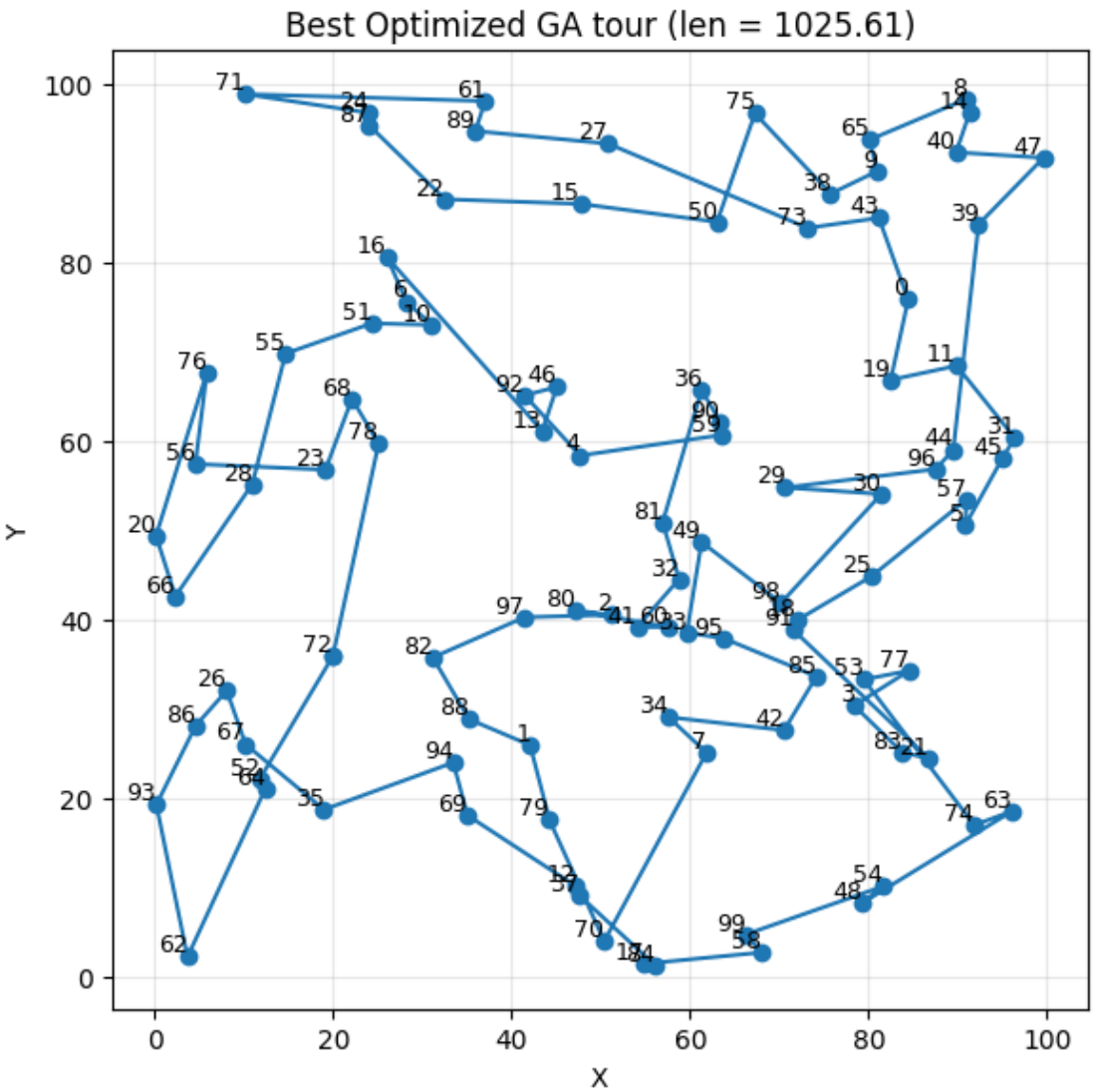
[ ]:
```
plt.figure(figsize=(6,4))
plt.plot(ga_best_costs_100)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("GA convergence on TSP(100 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(opt_ga_best_costs_100)
```

```
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("Optimized GA convergence on TSP(100 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_100, best_tour_ga_100, title=f"Best GA tour for 100␣
↪Cities(len = {best_len_ga_100:.2f})")
plot_best_tour(coords_100, opt_best_tour_ga_100, title=f"Best Optimized GA tour␣
↪for 100 Cities(len = {opt_best_len_ga_100:.2f})")
```




Optimized GA convergence on TSP
Best tour length
4500
4000
3500
3000
2500
2000
1500
1000
0
100
200
300
400
500
Generation

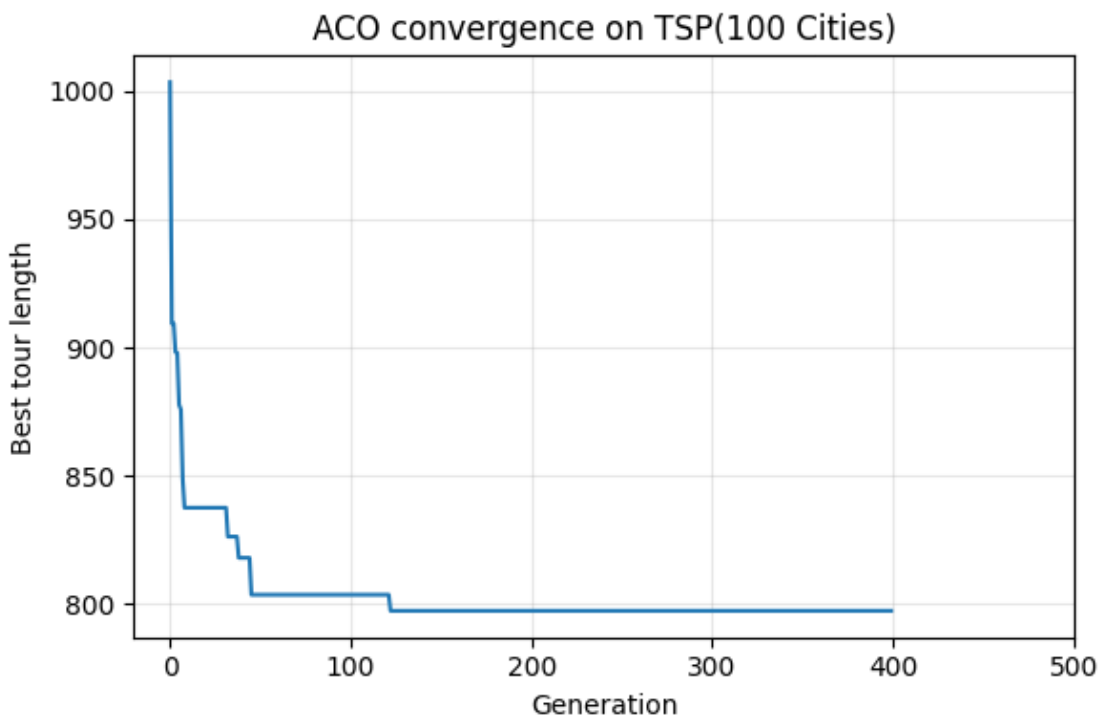

Best GA tour (len = 1602.06)
X
Y
0
20
40
60
80
100

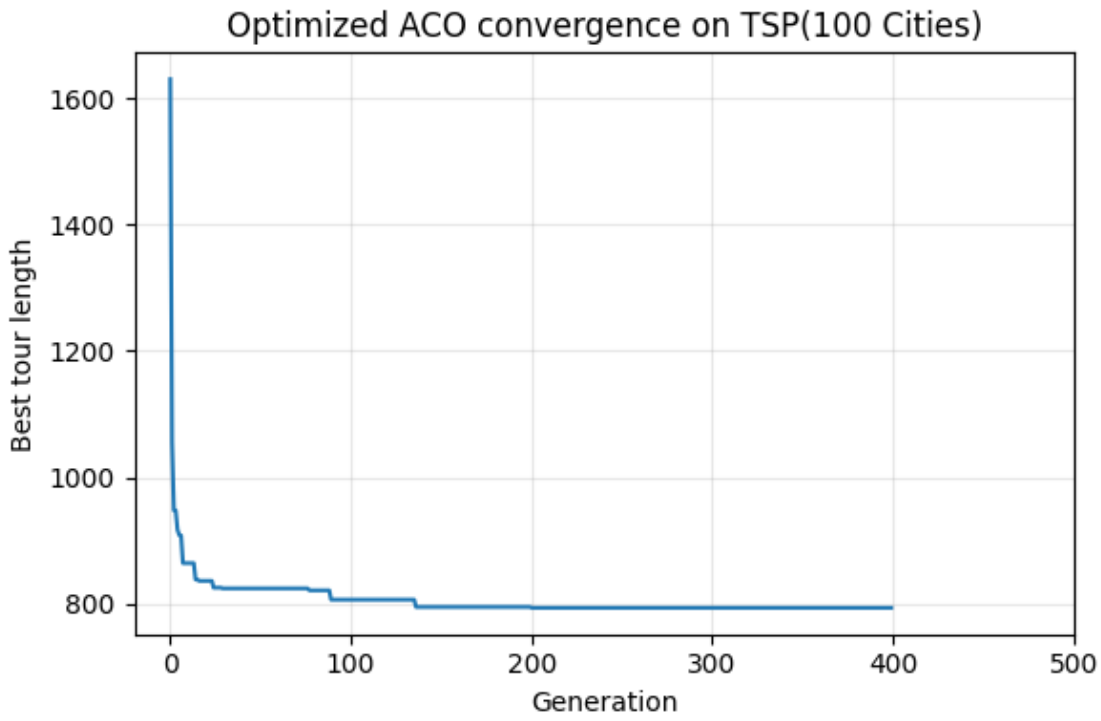


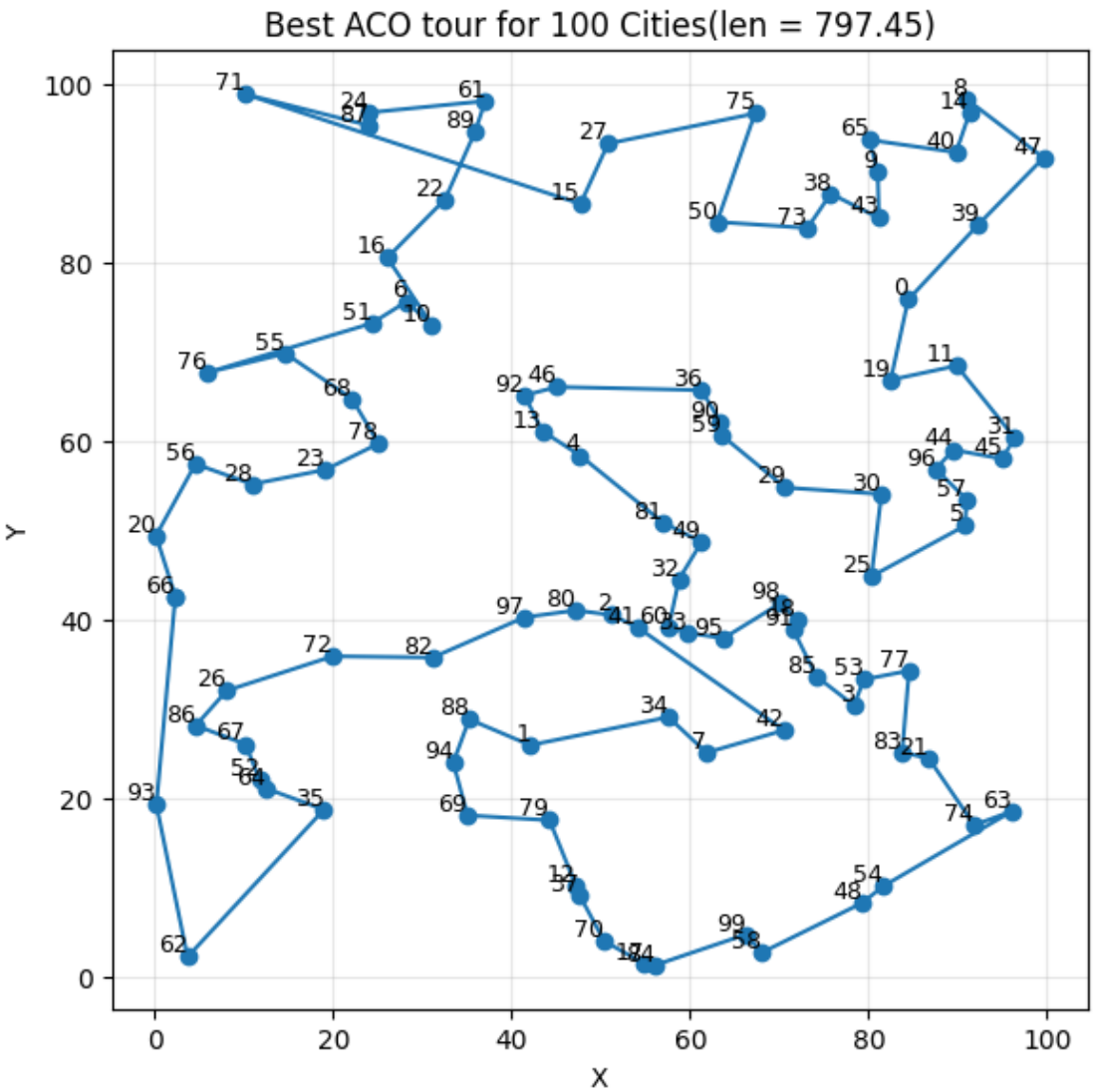
According to the Convergence graphs it can be seen that the Optimized GA has reached better solutions at earlier generations. As a result it converges faster and reach a better cost(tour length) within the iteration limit. Moreover, by comparing the tours of both original GA and optimized GA, we can see that the tour of Optimized GA has reduced the complexity of the tour path and finally reduced the total length.

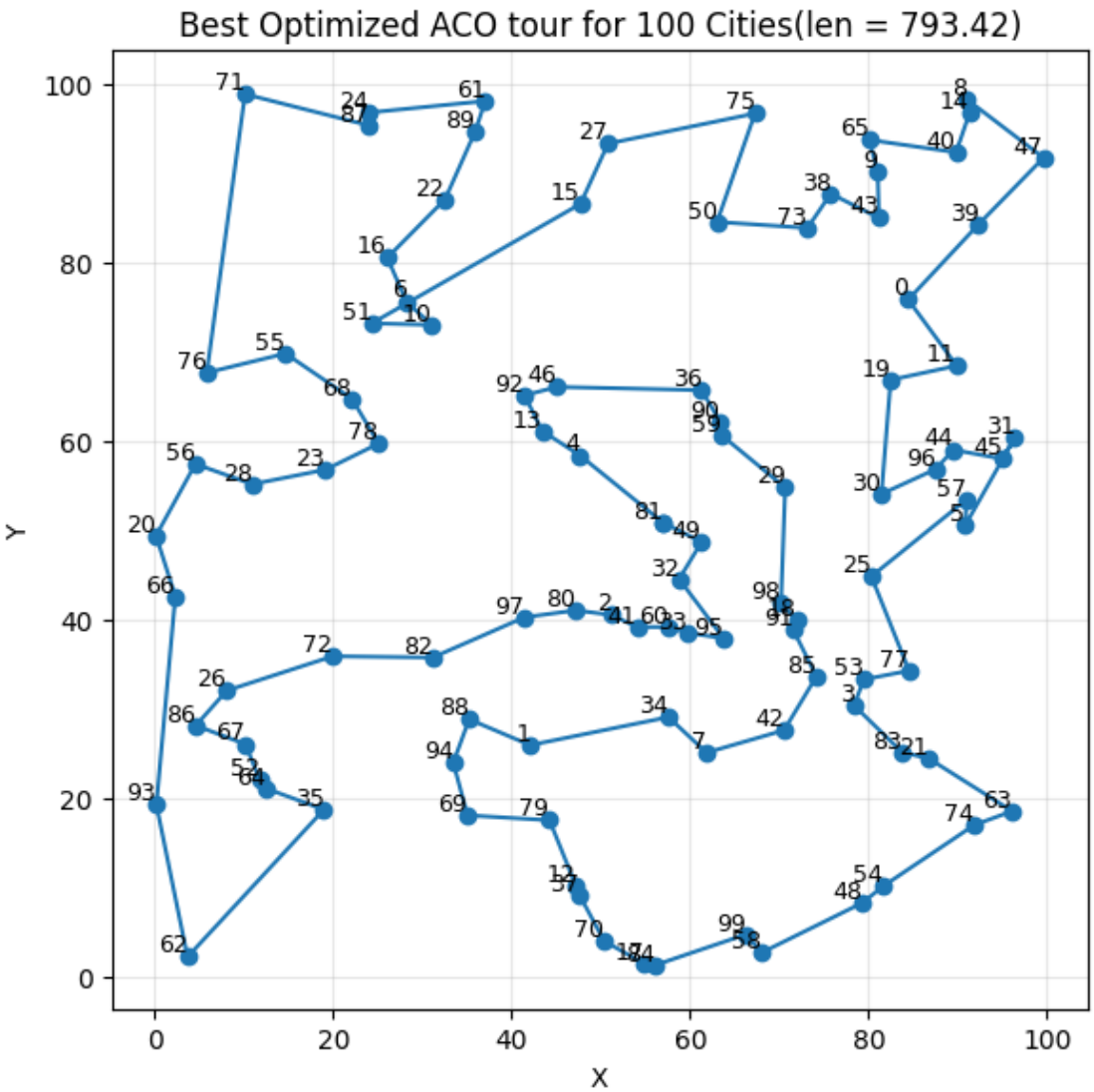
[ ]:
```
plt.figure(figsize=(6,4))
plt.plot(aco_best_costs_100)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("ACO convergence on TSP(100 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(opt_aco_best_costs_100)
plt.xlabel("Generation")
plt.ylabel("Best tour length")
plt.title("Optimized ACO convergence on TSP(100 Cities)")
plt.xlim(xmax=500)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plot_best_tour(coords_100, best_tour_aco_100, title=f"Best ACO tour for 100␣
↪Cities(len = {best_len_aco_100:.2f})")
plot_best_tour(coords_100, opt_best_tour_aco_100, title=f"Best Optimized ACO␣
↪tour for 100 Cities(len = {opt_best_len_aco_100:.2f})")
```




Optimized ACO convergence on TSP(100 Cities)
Best tour length
1600
1400
1200
1000
800
0
100
200
300
400
500
Generation


Best ACO tour for 100 Cities(len = 797.45)
X
Y
0
20
40
60
80
100

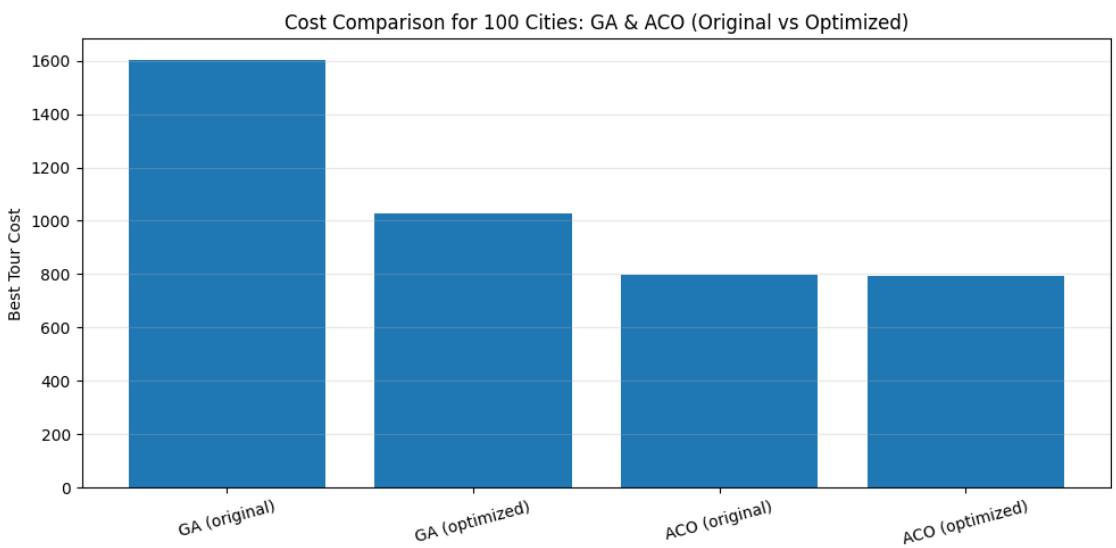


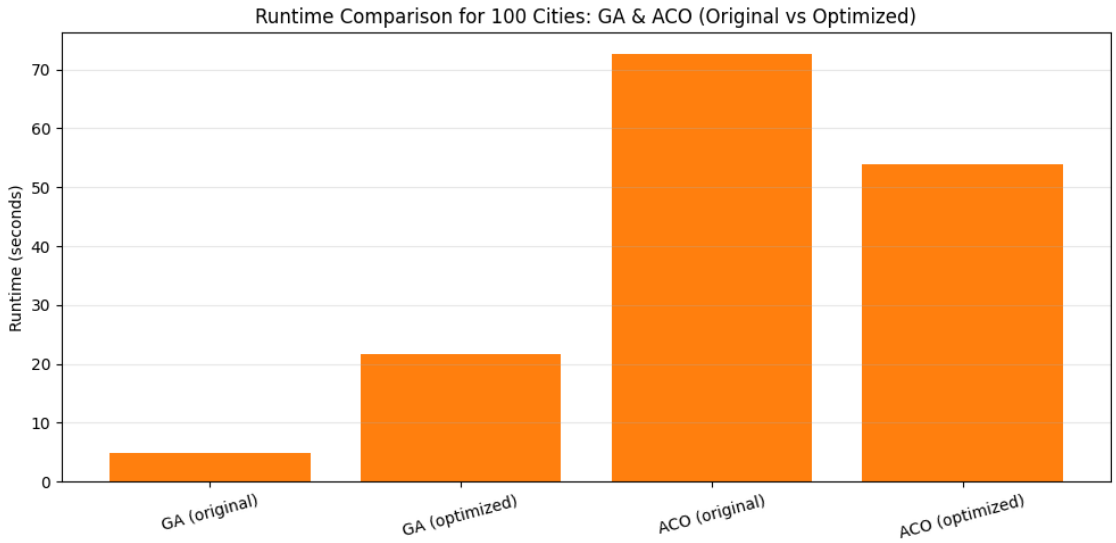
According to the Convergence graphs, it can be seen that the Optimized ACO has reached better solutions at earlier generations. As a result it converges faster and reach a slightly better cost(tour length) within the iteration limit. There is no significant difference between the tour path of both original and optimized ACO. However, Optimized ACO has reached a better final cost.

[ ]:
```
data = {
    "Algorithm": [
        "GA (Original)", "GA (Optimized)",
        "ACO (Original)", "ACO (Optimized)"
    ],
    "Best Cost": [
        best_len_ga_100, opt_best_len_ga_100,
        best_len_aco_100, opt_best_len_aco_100
```

```
plt.show()


#   PLOT 2 - TIME Comparison
plt.figure(figsize=(10,5))
plt.bar(x, times, color="tab:orange")
plt.xticks(x, labels, rotation=15)
plt.ylabel("Runtime (seconds)")
plt.title("Runtime Comparison for 100 Cities: GA & ACO (Original vs Optimized)")
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
         Algorithm    Best Cost  Runtime (sec)
0    GA (Original)  1602.059123       4.913047
1   GA (Optimized)  1025.609266      21.691031
2   ACO (Original)   797.454289      72.603525
3  ACO (Optimized)   793.415001      53.980039
```





From the above bar graph related to Cost(Tour length) comparison, it can be seen that both GA and ACO has improved it's final cost after optimization. Moreover, from the runtime comparison, it can be seen that, optimization has improved runtime efficiency of both GA and ACO as well.

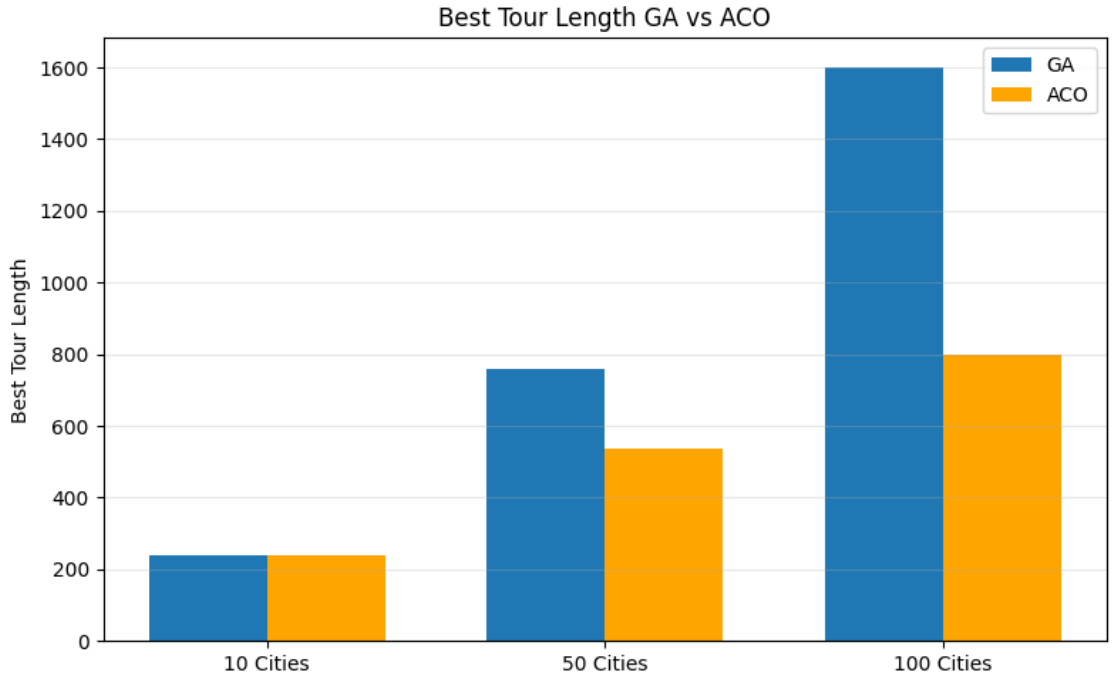
In General:

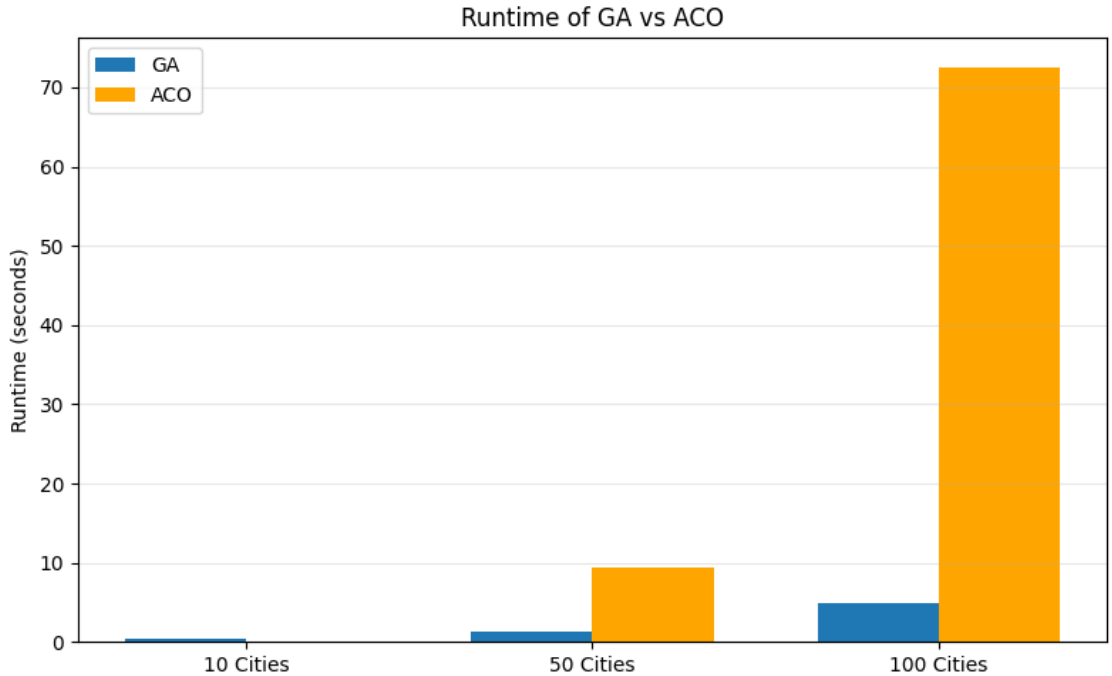
According to the results obtained after optimization, it can be concluded that, as the number of cities increases and the problem becomes complex, Hyper Parameter Optimization(HPO) becomes more applicable and useful.

6 Performance Comparison of GA(Original) and ACO(Original).

[ ]:
```
data = {
    "Algorithm": [
        "GA (10)",  "ACO (10)", "GA (50)",
        "ACO (50)", "GA (100)","ACO(100)"
    ],
    "Best Cost": [
        best_len_ga_10, best_len_aco_10, best_len_ga_50,
        best_len_aco_50, best_len_ga_100, best_len_aco_100
    ],
    "Runtime (sec)": [
        elapsed_GA_10,  elapsed_ACO_10, elapsed_GA_50,
        elapsed_ACO_50, elapsed_GA_100, elapsed_ACO_100
    ]
}

df = pd.DataFrame(data)
```

```
2   GA (50)    759.713257        1.259098
3  ACO (50)    536.256224        9.420384
4  GA (100)   1602.059123        4.913047
5  ACO(100)    797.454289       72.603525
```





According to the above Best Tour Length(=Cost) comparison of GA and ACO, when there are only 10 Cities in TSP problem, both GA and ACO has achieved the same final cost(Tour length). However the runtime efficiency of ACO is better.

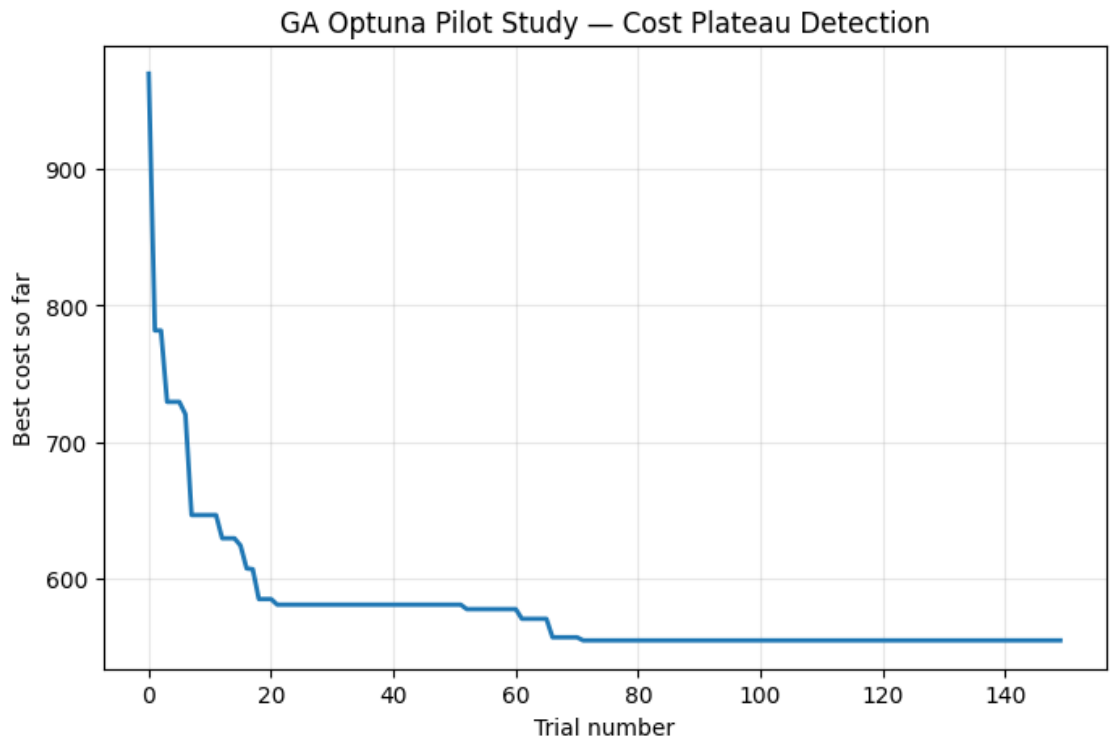
When there are 50 Cities, the best tour length of ACO is considerably better than GA. However, from the runtime comparison, it can be seen that the ACO has invested much time than GA in order to achieve the improved tour length.

When there are 100 cities, the final tour length achieved by ACO is significantly better than GA. However, ACO has invested a large amount of time in order to achieve that improved result.

7 Appendix

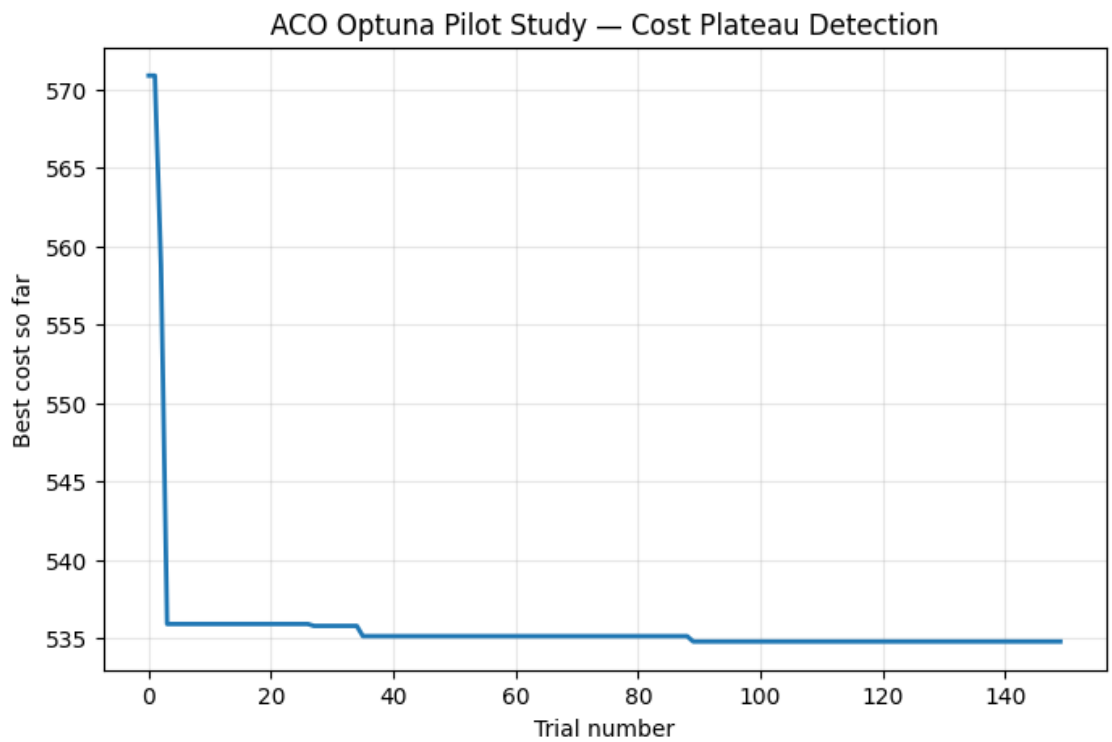
[ ]:
```
def obj_ga_pilot( trial: optuna.Trial,dist) -> float:
  pop_size     = trial.suggest_int("population_size", 20, 400, log=True)
  cx_rate      = trial.suggest_float("crossover_rate", 0.6, 1.0)
  mut_rate     = trial.suggest_float("mutation_rate", 1e-3, 0.3, log=True)
  elite_size   = trial.suggest_int("elitism", 0, max(1, pop_size // 10))
  tournament_k = trial.suggest_int("tournament_k", 2, min(10, pop_size))

  generations  = trial.suggest_int("generations", 50, 500, log=True)

  seeds = (0, 42, 777)
  costs= []
  for s in seeds:
      _, best_len,_= GA(
          dist,
          generations=generations,
          seed=s,
          pop_size=pop_size,
          cx_rate=cx_rate,
          mut_rate=mut_rate,
          elite_size=elite_size,
          tournament_k=tournament_k
      )


      costs.append(float(best_len))


  mean_cost = sum(costs) / len(costs)

  # For logging/analysis
  trial.set_user_attr("mean_cost", mean_cost)
```

```
parameters: {'population_size': 326, 'crossover_rate': 0.9997922083612262,
'mutation_rate': 0.19247132684336024, 'elitism': 23, 'tournament_k': 5,
'generations': 500}. Best is trial 71 with value: 554.848548445304.
[I 2025-11-23 07:56:21,954] Trial 149 finished with value: 576.4740687281009 and
parameters: {'population_size': 335, 'crossover_rate': 0.9735428458616127,
'mutation_rate': 0.18130724580836752, 'elitism': 30, 'tournament_k': 5,
'generations': 499}. Best is trial 71 with value: 554.848548445304.
```




```
Best cost found: 554.848548445304
```

[ ]:
```
coords = generate_TSP(50)           # or 10, 100 etc.
dist   = build_distance_matrix(coords)

study, pilot_curve = run_pilot("ACO",dist, n_trials=150)

plt.figure(figsize=(8,5))
plt.plot(pilot_curve, linewidth=2)
plt.xlabel("Trial number")
plt.ylabel("Best cost so far")
plt.title("ACO Optuna Pilot Study - Cost Plateau Detection")
plt.grid(True, alpha=0.3)
plt.show()
```




```
Best cost found: 534.8079213764594
```